INTERVIEW PREPARATION · RUST

Rust for Python Developers

The core language, read through working code

This book teaches enough Rust to read, change and discuss real code in an interview. Every chapter walks through a file in the companion project and shows the Python you would write for the same job. The last chapters are hands-on drills and a list of interview questions with short answers.

Study Guide · Generated 4 November 2026
Companion code: the `rust.tutorial` repository · Rust 2024 edition

Contents

1 **What Rust Is** 4

2 **The Companion Project** 7

3 **Syntax Basics** 12

4 **Types and Variables** 15

5 **Control Flow** 19

6 **Ownership and Borrowing** 22

7 **Structs and Methods** 27

8 **Enums and Pattern Matching** 32

9 **Error Handling: Result and Option** 37

10 **Collections** 41

11 **Traits and Generics** 44

12 **Iterators and Closures** 49

13 **Lifetimes and Smart Pointers** 53

14 **Concurrency and Async** 58

15 **Walkthrough: The Order System** 63

16 **Walkthrough: The Web API** 74

17 **Hands-on Drills** 79

18 **Interview Quick Reference** 83

A **Drill Solutions** 88

Audience and Scope

The reader writes Python and has not used Rust. The goal is working fluency for an interview: read unfamiliar Rust, make small changes, run the tests, and explain the core ideas in the right vocabulary. The goal is not to become a Rust developer overnight.

Every Rust listing labelled with a file path is copied from the companion project, which builds with no compiler or Clippy warnings, and all 12 of its tests pass. Program output shown in the book is the real output of `cargo run`, and every compiler error shown is the real output of `cargo build`.

If You Have Two Hours

- Read chapters 1, 6, 9, 11 and 15. Ownership (6) is the idea interviewers test most; the others cover how Rust replaces exceptions, classes and inheritance.
- Skim chapters 8 (enums) and 14 (concurrency).
- Do drills 1 to 3 in chapter 17 at a keyboard.
- Read chapter 18 last, out loud.

1 What Rust Is

Rust is a statically typed, compiled systems language. It started at Mozilla, reached version 1.0 in 2015, and is now run by the Rust Foundation. "Statically typed" means every variable's type is fixed and checked when the program is compiled, before it runs. "Compiled" means the compiler turns your source into a native executable for one kind of machine: there is no interpreter and no virtual machine.

Rust's selling point is that it is as fast as C and C++ but prevents their most common bugs at compile time: using memory after it was freed, freeing it twice, reading past the end of an array, and two threads changing the same data at once. It does this without a garbage collector, through a set of rules called **ownership** (chapter 6).

Python developers meet Rust more often than they realise. `uv`, `ruff`, `polars` and `pydantic-core` (the engine inside Pydantic 2) are written in Rust and called from Python through a library named **PyO3**. "Rewrite the hot loop in Rust" is now a common answer to "this Python is too slow".

1.1 The Pieces

The table lists the tools interviewers name, with the nearest Python equivalent of each.

Piece	What it is	Python analogue
rustc	The compiler. You rarely call it directly; Cargo does.	The CPython interpreter
Cargo	Build tool and package manager: create, build, run, test, add dependencies.	`uv`, `pip` and `pytest` together
crates.io	The public package registry. A package is a **crate**.	PyPI
rustup	Installs and switches Rust versions.	`pyenv`, `uv python`
std	The standard library: collections, files, threads, networking. Smaller than Python's; no HTTP, JSON or async runtime.	The standard library
Clippy, rustfmt	The linter and the formatter: `cargo clippy`, `cargo fmt`.	`ruff check`, `ruff format`

1.2 How a Rust Program Runs

`cargo build` runs the compiler, which checks every type, every borrow and every `match` in every file, and refuses to produce a program if any check fails. The output is a native executable in `target/debug/`. Running it needs nothing else installed: no Rust, no runtime, no interpreter.

Python finds a type mistake only when the faulty line runs. Rust finds it, and a whole class of memory and threading mistakes, before the program can start. The cost is a slower edit-compile cycle and a compiler that says no often. Rust developers describe this as "fighting the borrow checker" while learning, and "if it compiles, it usually works" afterwards.

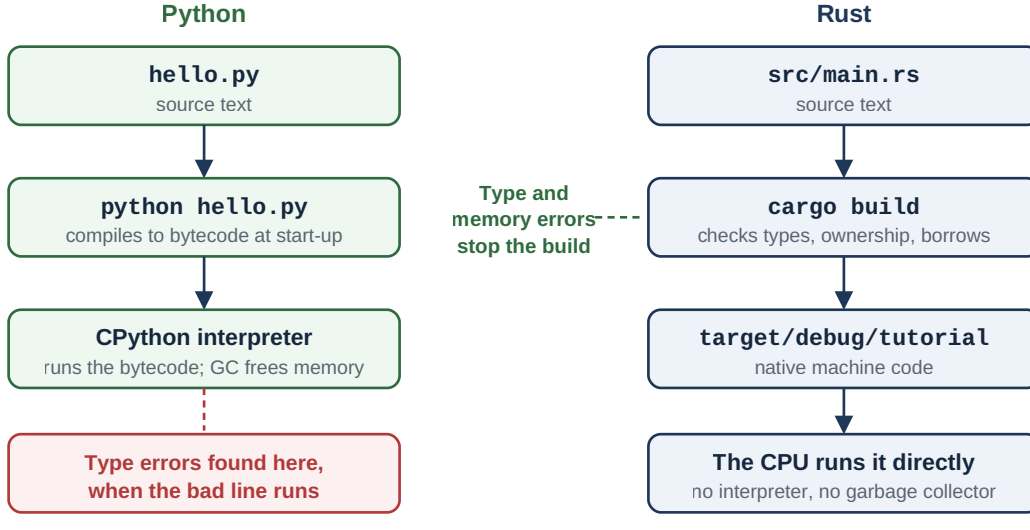


Figure 1. Python checks types while running; Rust checks types and memory use before running.

1.3 Editions and Releases

A new stable Rust version ships every six weeks, and old code keeps compiling. Larger changes arrive through **editions**: 2015, 2018, 2021 and 2024. Each crate names its edition in `Cargo.toml`, and crates of different editions work together. The companion project uses the 2024 edition. Code written for 2018 or 2021 looks almost the same; section 9.3 points out the one piece of 2024-only syntax the project uses.

1.4 Python and Rust Side by Side

Topic	Python	Rust
Typing	Dynamic; type hints are optional and not enforced	Static; every type checked at compile time, mostly inferred
Blocks	Indentation	`{ }` braces; statements end in `;`
Memory	Reference counting plus a cycle collector	Ownership: each value has one owner and is freed when the owner goes out of scope
Missing values	`None`, allowed anywhere	No null. `Option<T>` where a value may be missing
Errors	Exceptions	`Result<T, E>` return values; `panic!` for bugs
Objects	Classes with inheritance	Structs and enums with methods; traits instead of inheritance
Variables	Can always be reassigned	Immutable unless declared `mut`
Threads	Default CPython build has the GIL: one thread runs Python at a time	Threads run truly in parallel; data races are compile errors
Speed	Interpreted; slow for CPU-heavy work	Compiled to machine code; comparable to C and C++

2 The Companion Project

The companion project is a Cargo **workspace** with two **packages**. A package is a directory with a `Cargo.toml`; it builds into one library, one or more programs, or both. A workspace builds several packages together and shares one lock file and one `target/` directory. A Python monorepo with several `pyproject.toml` files managed by one `uv` workspace is the closest match.

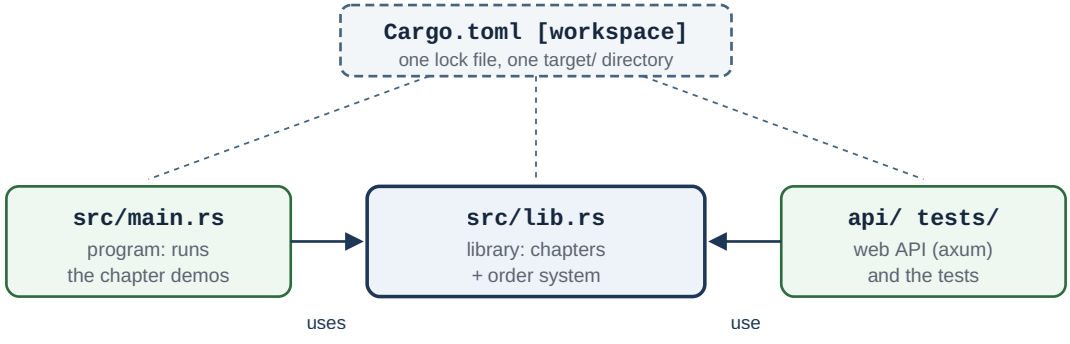


Figure 2. The workspace. The demo program, the web API and the tests all use the library in `src/lib.rs`.

2.1 Files

Path	Contents
`Cargo.toml`	The `tutorial` package and the workspace that adds `api/`.
`src/main.rs`	Entry point. Runs one chapter's demo or all of them.
`src/lib.rs`	The library root: lists every module.
`src/ch03_basics.rs` … `ch14_concurrency.rs`	One file per book chapter. The number in the file name is the chapter number.
`src/orders/`	The order system: products, orders, discounts, checkout.
`src/ch15_orders.rs`	Runs the order system end to end.
`src/money.rs`	Formats amounts held as whole cents.
`tests/`	Integration tests for `BankAccount` and `OrderService`.
`api/`	The `tutorial-api` package: a web API over the order system.

Path	Contents
`Cargo.lock`	Exact versions of every dependency (Python: `uv.lock`). Committed.

2.2 The cargo Command

One command, `cargo`, replaces the separate Python tools. Run these from the repo root. If `cargo` is not found, install Rust with rustup (see the README) and open a new terminal.

Task	Python	Rust
Run all chapters	`python main.py`	`cargo run`
Run chapter 6	`python main.py 6`	`cargo run -- 6`
Type-check only (fast)	`mypy .`	`cargo check`
Compile	(none)	`cargo build`
Run tests	`pytest`	`cargo test`
Lint	`ruff check`	`cargo clippy`
Format code	`ruff format`	`cargo fmt`
Add a package	`uv add requests`	`cargo add reqwest`
New project	`uv init`	`cargo new my_app`
Optimised build	(none)	`cargo build --release`
Virtual env	`.venv`	None needed. Dependencies are per project.

The demo numbers match this book's chapter numbers: `cargo run -- 6` runs the code for chapter 6, and the valid numbers are 3 to 15. The `--` separates Cargo's own options from the arguments passed to your program. `cargo run -p tutorial-api` runs the other package, the web API. Debug builds (the default) compile quickly and run slowly; `--release` is the reverse.

2.3 Cargo.toml

`Cargo.toml` lists a package's name, edition and dependencies. It is the counterpart of `pyproject.toml`. The root one, without its comments:

Cargo.toml

```
[package]
name = "tutorial"
version = "0.1.0"
edition = "2024"
rust-version = "1.88"

[dependencies]
serde = { version = "1.0.229", features = ["derive"] }
tokio = { version = "1.54.1", features = ["rt-multi-thread", "macros", "time"] }

[workspace]
members = ["api"]
```

- `edition`: which edition's rules this crate follows (section 1.3).
- `rust-version`: the oldest Rust that can build it (Python: `requires-python`). An older compiler stops with a message naming the version needed.
- `[dependencies]`: crates from crates.io. `"1.0.229"` means "1.0.229 or any later 1.x version" (Python: `>=1.0.229,<2`); `Cargo.lock` records the exact version chosen.
- `features`: optional parts of a crate to switch on, like Python extras (`pip install "fastapi[all]"`). `serde`'s `derive` feature is used in chapter 15; `tokio` is used in chapter 14.
- `[workspace]`: adds `api/` as a second package.

2.4 Modules: lib.rs and main.rs

A Rust package can hold a **library** (`src/lib.rs`) and a **program** (`src/main.rs`). The program uses the library like any outside code would. Each `pub mod name;` line in `lib.rs` tells the compiler to include the file `src/name.rs` (or `src/name/mod.rs`) as a **module**. A module is Python's module: a namespace, one per file. Unlike Python, files are not found automatically; a file nobody declares with `mod` is not compiled.

src/lib.rs

```
pub mod ch03_basics;
pub mod ch04_types;
pub mod ch05_control_flow;
pub mod ch06_ownership;
pub mod ch07_structs;
pub mod ch08_enums;
pub mod ch09_errors;
pub mod ch10_collections;
pub mod ch11_traits;
pub mod ch12_iterators;
pub mod ch13_lifetimes;
pub mod ch14_concurrency;
pub mod ch15_orders;
pub mod money;
pub mod orders;
```

`pub` makes an item public. Everything in Rust is private to its module unless marked `pub`, and the compiler enforces it. Python's leading underscore is only a convention.

2.5 main.rs Walkthrough

`main.rs` reads the command line and calls each chapter's `run` function. It uses syntax from the next few chapters; skim it now and return to it after chapter 6. Execution starts at `fn main()`, as Python starts at `if __name__ == "__main__":`.

src/main.rs

```
use tutorial::{
    ch03_basics, ch04_types, ch05_control_flow, ch06_ownership, ch07_structs,
ch08_enums,
    ch09_errors, ch10_collections, ch11_traits, ch12_iterators, ch13_lifetimes,
ch14_concurrency,
    ch15_orders,
};

// in main():
// env::args() is Python's sys.argv; skip(1) drops the program name
let args: Vec<String> = env::args().skip(1).collect();
let mut chapters: Vec<u32> = Vec::new();
if args.is_empty() {
    for (number, _) in titles {
        chapters.push(number);
    }
} else {
    for arg in &args {
        chapters.push(arg.parse().expect("chapter must be a number"));
    }
}

for chapter in chapters {
    let mut title = "unknown";
```

```
    for (number, name) in titles {
        if number == chapter {
            title = name;
        }
    }
    println!("\n=== Chapter {chapter}: {title} ===");
    match chapter {
        3 => ch03_basics::run(),
        4 => ch04_types::run(),
        5 => ch05_control_flow::run(),
        6 => ch06_ownership::run(),
        7 => ch07_structs::run(),
        8 => ch08_enums::run(),
        9 => ch09_errors::run(),
        10 => ch10_collections::run(),
        11 => ch11_traits::run(),
        12 => ch12_iterators::run(),
        13 => ch13_lifetimes::run(),
        14 => ch14_concurrency::run(),
        15 => ch15_orders::run(),
        _ => println!("No demo for chapter {chapter}. Valid chapters: 3 to 15."),
    }
}
```

- `use tutorial::{ch03_basics, ...};` imports modules from the library. It is Python's `from tutorial import ch03_basics, ...`. `::` is Rust's path separator, where Python uses `.`.
- `Vec<String>` is a growable list of strings. `collect()` gathers the arguments into it (chapter 12).
- `arg.parse()` converts text to a number. It returns a `Result` (chapter 9), and `.expect(...)` stops the program with that message if the text is not a number.
- `match chapter` picks the branch for the number; `_` matches anything else (chapter 5). There is no fall-through between arms, so there is no `break`.
- `println!` ends in `!` because it is a **macro**: code that generates code at compile time. It checks the format string against its arguments while compiling.

3 Syntax Basics

This chapter covers what every Rust file looks like: functions, variables, printing and blocks. File: `src/ch03_basics.rs`. Run it with `cargo run -- 3`.

3.1 The Whole File

src/ch03_basics.rs

```rust
pub fn run() {
    // let makes a variable. Types are inferred: year is an i32.
    let name = "Ada";
    let year = 1843;
    println!("{name} published the first program in {year}.");

    // Variables are immutable unless declared with mut
    let mut count = 0;
    count += 1;
    println!("count is {count}");

    // {:.2} is Python's :.2f, {:>8} right-aligns, {:?} is debug output (like
repr)
    let price = 19.987;
    println!("price {price:.2}, padded [{year:>8}], debug {name:?}");

    // format! builds a String instead of printing it (Python: an f-string)
    let greeting = format!("Hello, {name}!");
    // len() counts bytes, not characters (they match for plain ASCII text)
    println!("{greeting} is {} bytes long", greeting.len());

    // Shadowing: a new let with the same name replaces the old variable
    let year = year + 1;
    println!("a year later it was {year}");

    // A block is an expression: its last line, with no semicolon, is its value
    let area = {
        let width = 3;
        let height = 4;
        width * height
    };
    println!("area is {area}");

    println!("square(7) is {}", square(7));
}

// Parameter and return types are required. The last expression is returned.
fn square(n: i32) -> i32 {
    n * n
}
```

Output

```
Ada published the first program in 1843.
count is 1
price 19.99, padded [    1843], debug "Ada"
Hello, Ada! is 11 bytes long
a year later it was 1844
area is 12
square(7) is 49
```

3.2 Reading It

- `fn` defines a function (Python: `def`). `pub` makes it callable from other modules; `square` has no `pub`, so only this file can call it.
- `let` creates a variable. Its type is inferred from the value, but it is still fixed: `year` is an `i32` (32-bit integer) forever.
- **Immutable by default.** A plain `let` variable cannot be changed. `let mut` allows changes. Python has no equivalent; the closest is a naming convention for constants.
- **Formatting.** `println!("{name}")` is Python's `print(f"{name}")`. Format specs follow a colon, as in Python: `{price:.2}`, `{year:>8}`. `{:?}` prints the *debug* form, which for a string includes the quotes, like `repr()`. `{}` with an argument after the string fills in an expression.
- **Shadowing.** `let year = year + 1;` makes a new variable that hides the old one. It is common in Rust, for example to convert a value and keep the same name.
- **Blocks are expressions.** The value of a `{ }` block is its last expression, written *without* a semicolon. `square` returns `n * n` this way. A `return` keyword exists, but is used only to leave a function early.
- **Types in signatures are required.** `fn square(n: i32) -> i32` is Python's `def square(n: int) -> int`, except the compiler enforces it.

The semicolon matters. `n * n` is a value; `n * n;` is a statement that throws the value away. Adding a stray semicolon to the last line of a function is a common beginner error; drill 2 shows the message it produces.

3.3 Comments and Naming

Kind	Syntax	Python
Line comment	`// text`	`# text`
Doc comment on an item	`/// text`	A docstring

Kind	Syntax	Python
Doc comment on a module	`//! text`	A module docstring
Functions, variables, modules	`snake_case`	`snake_case`
Types and traits	`CamelCase`	`CamelCase`
Constants	`const MAX: u32 = 10;`	`MAX = 10`

The naming rules are the same as Python's, and the compiler warns when you break them. `cargo doc --open` turns doc comments into a website.

4 Types and Variables

Rust has more number types than Python, two string types, and no `None` that can appear anywhere. File: `src/ch04_types.rs`.

4.1 Numbers, Booleans and Characters

src/ch04_types.rs

```rust
// in run():
// Scalar types. The annotations are optional here; they show the names.
let age: i32 = 36;
let population: i64 = 8_100_000_000;
let ratio: f64 = 0.75;
let ready: bool = true;
let letter: char = 'R';
println!("{age} {population} {ratio} {ready} {letter}");

// Integer division truncates. 7 / 2.0 would not compile: no mixed types.
println!(
    "7 / 2 = {}, 7.0 / 2.0 = {}, 7 % 2 = {}",
    7 / 2,
    7.0 / 2.0,
    7 % 2
);

// No automatic conversion between number types; `as` converts explicitly
let scaled = age as f64 * ratio;
println!("scaled = {scaled}");

// Integers have a fixed size. A u8 holds 0 to 255.
let small: u8 = 250;
println!("checked_add: {:?}", small.checked_add(10));
println!("wrapping_add: {}", small.wrapping_add(10));
```

Output

```
36 8100000000 0.75 true R
7 / 2 = 3, 7.0 / 2.0 = 3.5, 7 % 2 = 1
scaled = 27
checked_add: None
wrapping_add: 4
```

Rust type	Holds	Python
`i8 i16 i32 i64 i128`	Signed integers of that many bits. `i32` is the default.	`int`
`u8 u16 u32 u64 u128`	Unsigned integers (zero and up). `u8` is a byte.	`int`

Rust type	Holds	Python
`usize, isize`	Pointer-sized integers. Lengths and indexes are `usize`.	`int`
`f64, f32`	Floating point. `f64` is the default.	`float`
`bool`	`true` or `false` (lower case)	`bool`
`char`	One Unicode character, in single quotes: `'R'`	(a 1-character `str`)

- **No mixing.** `age * ratio` does not compile, because one is an `i32` and the other an `f64`. Python would convert silently. `age as f64` converts explicitly.
- **Integer division truncates**: `7 / 2` is 3. There is no separate `//` operator.
- **Overflow.** Python integers grow without limit. A Rust `u8` holds 0 to 255. In a debug build, arithmetic that overflows stops the program with a panic; in a release build it wraps around. When overflow is possible, say what you want: `checked_add` returns `None` on overflow, `wrapping_add` wraps (250 + 10 = 4).
- `8_100_000_000`: underscores in numbers are ignored, as in Python.

4.2 Two String Types

src/ch04_types.rs

```
// in run():
// Two string types. String owns its text and can grow;
// &str ("string slice") borrows text that something else owns.
let mut owned: String = String::from("hello");
owned.push_str(", world");
let borrowed: &str = &owned;
let literal: &str = "literals are &str";
println!("{borrowed} / {literal}");
println!(
    "upper: {}, words: {}",
    owned.to_uppercase(),
    owned.split(", ").count()
);
```

Output

```
hello, world / literals are &str
upper: HELLO, WORLD, words: 2
```

This confuses every newcomer, and interviewers ask about it.

- **`String`** owns its text, stored on the heap. It can grow (`push_str`). Create one with `String::from("...")`, `"...".to_string()` or `format!`.
- **`&str`** ("string slice", said "stir" or "string ref") is a borrowed view of text owned by something else: a `String`, or the program file itself for a literal such as `"world"`. It cannot grow.
- Function parameters are usually `&str`, because a `&String` converts to a `&str` automatically, so the function accepts both. Struct fields are usually `String`, because the struct should own its data.
- Both are always valid UTF-8. `s[0]` does not compile, because one character can be several bytes. Use `s.chars()` to walk characters.

Chapter 6 explains the words "own" and "borrow". Python has one `str` type because its garbage collector decides when text is freed; in Rust, the type says who is responsible.

4.3 Tuples and Arrays

src/ch04_types.rs

```
// in run():
// Tuples group values of different types; arrays have a fixed length
let point: (i32, f64) = (3, 4.5);
let (x, y) = point;
let primes: [i32; 4] = [2, 3, 5, 7];
println!(
    "x={x} y={y} first prime={} count={}",
    primes[0],
    primes.len()
);
```

Output

```
x=3 y=4.5 first prime=2 count=4
```

Tuples work as in Python, including unpacking with `let (x, y) = point;`. Access one field with `point.0`. An array `[i32; 4]` has a fixed length that is part of its type; the growable list is `Vec` (chapter 10). Indexing past the end of either stops the program with a panic instead of reading random memory.

4.4 No Null: Option

src/ch04_types.rs

```
// in run():
// No null. A value that may be missing has type Option<T>: Some(value) or None.
// find returns the position of a character, if the text contains it.
let w_at: Option<usize> = owned.find('w');
let z_at: Option<usize> = owned.find('z');
println!("{w_at:?} {z_at:?}");
println!(
    "z found? {}, z position or 0: {}",
    z_at.is_some(),
    z_at.unwrap_or(0)
);
```

Output

```
Some(7) None
z found? false, z position or 0: 0
```

Rust has no `None`/`null` that any variable might hold. A value that may be missing has the type `Option<T>`, which is either `Some(value)` or `None`. `Option<usize>` and `usize` are different types, so you cannot use a possibly-missing number as if it were present; the compiler makes you handle `None` first. Python's `Optional[int]` hint says the same thing, but nothing enforces it. Chapter 8 shows how to take an `Option` apart with `match`; chapter 9 shows its helper methods.

5 Control Flow

Rust has `if`, `match`, and three loops: `for`, `while` and `loop`. Conditions need no parentheses, but the braces are required. File: `src/ch05_control_flow.rs`.

5.1 if Is an Expression

src/ch05_control_flow.rs

```
// in run():
// if is an expression, so it produces a value (Python: a if cond else b)
let temperature = 23;
let feel = if temperature > 25 {
    "hot"
} else if temperature > 15 {
    "mild"
} else {
    "cold"
};
println!("{temperature} degrees is {feel}");
```

Because `if` produces a value, Rust needs no separate conditional expression; this is Python's `"hot" if t > 25 else ...`. Every branch must produce the same type. The condition must be a `bool`: `if count` with a number does not compile, where Python would test for non-zero.

5.2 match

src/ch05_control_flow.rs

```
// in run():
for score in [95, 82, 71, 40] {
    println!("{score} -> {}", grade(score));
}

// match must cover every possible value; the compiler checks
fn grade(score: u32) -> char {
    match score {
        90..=100 => 'A',
        80..=89 => 'B',
        70..=79 => 'C',
        _ => 'F',
    }
}
```

Output

```
95 -> A
82 -> B
71 -> C
40 -> F
```

`match` compares a value against **patterns**, top to bottom, and runs the first arm that fits. `90..=100` is an inclusive range; `_` matches anything. Python 3.10's `match` statement is the closest equivalent, but Rust's is checked: if the arms do not cover every possible value, the program does not compile. Delete the `_` arm and the compiler lists the numbers you forgot. Chapter 8 uses `match` on enums, where this check matters most.

5.3 Loops

src/ch05_control_flow.rs

```rust
// in run():
// 1..=5 includes 5; 1..5 does not (Python: range(1, 6) and range(1, 5))
let mut total = 0;
for n in 1..=5 {
    total += n;
}
println!("sum of 1..=5 is {total}");

// enumerate gives (index, item) pairs, as in Python
let languages = ["Python", "Rust", "SQL"];
for (i, language) in languages.iter().enumerate() {
    println!("{i}: {language}");
}

let mut n = 27;
let mut steps = 0;
while n != 1 {
    n = if n % 2 == 0 { n / 2 } else { 3 * n + 1 };
    steps += 1;
}
println!("27 reaches 1 in {steps} steps");

// loop repeats until break, and break can hand back a value
let mut guess = 0;
let first_big_square = loop {
    guess += 1;
    if guess * guess > 50 {
        break guess;
    }
};
println!("first number whose square passes 50: {first_big_square}");
```

Output

```
sum of 1..=5 is 15
0: Python
1: Rust
2: SQL
27 reaches 1 in 111 steps
first number whose square passes 50: 8
```

Python	Rust
`for n in range(1, 6):`	`for n in 1..=5 {`
`for n in range(5):`	`for n in 0..5 {`
`for i, x in enumerate(xs):`	`for (i, x) in xs.iter().enumerate() {`
`while n != 1:`	`while n != 1 {`
`while True:`	`loop {`
`break / continue`	`break / continue`

`loop` is an endless loop that can return a value through `break value;`. Rust has no C-style `for (i = 0; i < n; i++)`, and no `++` operator; write `i += 1`.

6 Ownership and Borrowing

Ownership is the idea that makes Rust different from every language you know, and the topic an interviewer is most likely to probe. It replaces the garbage collector. File: `src/ch06_ownership.rs`.

6.1 The Three Rules

1. Each value has one **owner**: the variable (or struct field, or collection) that holds it.
2. There is only one owner at a time. Assigning or passing the value **moves** ownership.
3. When the owner goes out of scope (its closing `}`), the value is **dropped**: its memory is freed, immediately and deterministically.

In Python, `b = a` makes two names for one object, and the garbage collector frees the object some time after the last name disappears. In Rust, `let b = a;` hands the value to `b`, and `a` may not be used again. Because exactly one owner exists, the compiler knows exactly where to free the memory, and no garbage collector is needed.

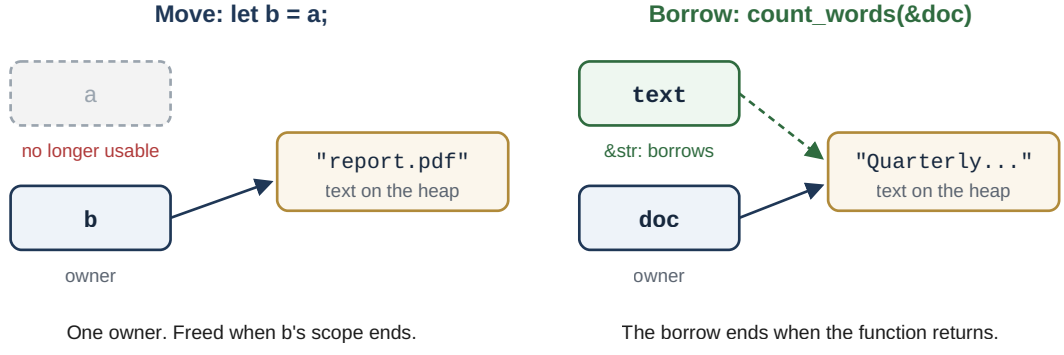


Figure 3. A move transfers the owner; a borrow lends access while the owner keeps the value.

6.2 Moves, Clones and Copies

src/ch06_ownership.rs

```rust
// in run():
// 1. Move: assigning a String moves it. The old name can no longer be used.
let a = String::from("report.pdf");
let b = a;
// println!("{a}");   // error[E0382]: borrow of moved value: `a`
println!("b = {b}");

// 2. clone makes a real copy when you want two independent values
let c = b.clone();
println!("b = {b}, c = {c}");

// 3. Small fixed-size types (numbers, bool, char) are Copy: copied, not moved
let x = 5;
let y = x;
println!("x = {x}, y = {y}");

// 4. Passing a value to a function moves it into the function
let title = String::from("Annual Report");
let length = take_ownership(title);
// println!("{title}");  // error[E0382]: title was moved
println!("the function took a title of {length} bytes");
```

src/ch06_ownership.rs

```rust
// Takes ownership of `text`; the String is freed when this function returns.
fn take_ownership(text: String) -> usize {
    text.len()
}
```

Output

```
b = report.pdf
b = report.pdf, c = report.pdf
x = 5, y = 5
the function took a title of 13 bytes
```

Move. After `let b = a;`, using `a` is a compile error. Remove the `//` in front of `println!("{a}")` and `cargo build` reports:

cargo build

```
error[E0382]: borrow of moved value: `a`
 --> src/ch06_ownership.rs:7:16
  |
5 |     let a = String::from("report.pdf");
  |         - move occurs because `a` has type `String`, which does not implement
the `Copy` trait
6 |     let b = a;
  |             - value moved here
7 |     println!("{a}");
  |                ^ value borrowed here after move
  |
help: consider cloning the value if the performance cost is acceptable
  |
6 |     let b = a.clone();
  |              ++++++++
```

- **Clone.** `.clone()` makes an independent deep copy (Python: `copy.deepcopy`). It costs time and memory, so Rust never does it implicitly.
- **Copy.** Small fixed-size types (integers, floats, `bool`, `char`, and tuples of them) implement the `Copy` trait. Assigning them copies the bits and both variables stay usable, as `x` and `y` show.
- **Passing to a function moves too.** `take_ownership(title)` moves the `String` into the function, which frees it when it returns. Afterwards `title` is unusable in `run`.

Read the compiler error above line by line. Rust's errors name the rule, point at each line involved and often suggest the fix. `rustc --explain E0382` prints a longer explanation with examples.

6.3 Borrowing

Moving everything into functions would be impractical, so Rust lets you **borrow**: create a **reference** that gives access to a value without taking ownership.

src/ch06_ownership.rs

```
// in run():
// 5. Borrow: &doc lends read access; doc keeps ownership
let doc = String::from("Quarterly Report for Q3");
let words = count_words(&doc);
println!("'{doc}' has {words} words");

// 6. Mutable borrow: &mut log lends write access
let mut log: Vec<String> = Vec::new();
add_entry(&mut log, "started");
add_entry(&mut log, "finished");
println!("log = {log:?}");
```

src/ch06_ownership.rs

```
// Borrows read-only. &str accepts a &String as well as a literal.
fn count_words(text: &str) -> usize {
    text.split_whitespace().count()
}

// Borrows mutably, so it may change the caller's vector.
fn add_entry(log: &mut Vec<String>, entry: &str) {
    log.push(entry.to_string());
}
```

Output

```
'Quarterly Report for Q3' has 4 words
log = ["started", "finished"]
```

- `&doc` creates a **shared reference** (type `&String`, accepted as `&str`). The function can read but not change the value.
- `&mut log` creates a **mutable reference**. The function can change the caller's vector. Both the variable (`let mut log`) and the borrow (`&mut`) must say `mut`, so a reader can see every place a value might change.
- When the function returns, the borrow ends and the owner has full control again.

In Python every argument is passed by reference and any function can change a mutable argument. In Rust the signature says which: `T` (takes ownership), `&T` (reads), `&mut T` (changes).

6.4 The Borrow Rule

At any moment, a value may have **either** any number of shared references **or** exactly one mutable reference, never both. And no reference may outlive the value it points to.

src/ch06_ownership.rs

```
// in run():
// 7. The borrow rule: any number of readers, or one writer, never both at once
let mut scores = vec![10, 20, 30];
let first = &scores[0];
// scores.push(40);  // error[E0502]: cannot borrow `scores` as mutable
//                   // because it is also borrowed as immutable
println!("first = {first}");
scores.push(40); // allowed: `first` is not used after this point
println!("scores = {scores:?}");
```

Output

```
first = 10
scores = [10, 20, 30, 40]
```

Why the rule exists: `push` may move the vector's contents to a bigger block of memory. If that happened while `first` still pointed into the old block, `first` would point at freed memory. In C++ that is a real, common bug. Rust rejects the program instead. Move the `push` above the `println!` that uses `first`, and the compiler says:

cargo build

```
error[E0502]: cannot borrow `scores` as mutable because it is also borrowed as
immutable
  --> src/ch06_ownership.rs:39:5
   |
38 |     let first = &scores[0];
   |                 ------ immutable borrow occurs here
39 |     scores.push(40);
   |     ^^^^^^^^^^^^^^^ mutable borrow occurs here
40 |     //                  // because it is also borrowed as immutable
41 |     println!("first = {first}");
   |                        ----- immutable borrow later used here
```

A borrow lasts only until its last use, not to the end of the block, so the final `push` in the file is allowed. The part of the compiler that checks these rules is the **borrow checker**.

In one sentence. Ownership means every value has one owner that frees it at the end of its scope; borrowing lends access through references, with many readers or one writer at a time, and the compiler checks all of it, so Rust is memory-safe without a garbage collector.

7 Structs and Methods

Rust has no classes. A **struct** holds data, and an `impl` block attaches methods to it. Together they do most of what a Python class does, without inheritance (chapter 11 covers what replaces it). File: `src/ch07_structs.rs`.

7.1 A Struct: BankAccount

`BankAccount` shows **encapsulation**: the data is private, and outside code can change it only through methods that check each change. Money is held as whole cents in an `i64`, never as a float; `120_00` is 120.00 written so the cents stand out.

src/ch07_structs.rs

```
/// A bank account. The fields are private, so code outside this module
/// can change the balance only through the methods below.
#[derive(Debug)]
pub struct BankAccount {
    id: String,
    owner: String,
    balance_cents: i64,
}

impl BankAccount {
    /// An associated function (no self). `new` is the constructor by convention.
    pub fn new(id: &str, owner: &str) -> BankAccount {
        BankAccount {
            id: id.to_string(),
            owner: owner.to_string(),
            balance_cents: 0,
        }
    }

    // &self: the method only reads the account
    pub fn balance(&self) -> i64 {
        self.balance_cents
    }

    pub fn owner(&self) -> &str {
        &self.owner
    }

    // &mut self: the method changes the account
    pub fn deposit(&mut self, cents: i64) -> Result<(), String> {
        if cents <= 0 {
            return Err(format!("deposit must be positive, got {cents}"));
        }
        self.balance_cents += cents;
        Ok(())
    }
```

```
    pub fn withdraw(&mut self, cents: i64) -> Result<(), String> {
        if cents > self.balance_cents {
            return Err(format!(
                "insufficient funds: balance {}, requested {}",
                dollars(self.balance_cents),
                dollars(cents)
            ));
        }
        self.balance_cents -= cents;
        Ok(())
    }

    pub fn summary(&self) -> String {
        format!(
            "{} {}: {}",
            self.id,
            self.owner,
            dollars(self.balance_cents)
        )
    }
}
```

The same class in Python, for comparison:

Python

```
class BankAccount:
    def __init__(self, id: str, owner: str):
        self.id = id
        self.owner = owner
        self._balance_cents = 0

    @property
    def balance(self) -> int:
        return self._balance_cents

    def deposit(self, cents: int) -> None:
        if cents <= 0:
            raise ValueError(f"deposit must be positive, got {cents}")
        self._balance_cents += cents
```

7.2 Reading the Struct

- **Fields** are declared once, with types, in the `struct`. Without `pub` they are private to this module. Python's `_balance` is a convention; Rust's privacy is enforced.
- **`impl BankAccount { ... }`** holds the methods. A type can have several `impl` blocks.
- **`new`** is an **associated function**: it has no `self`, so it is called on the type, `BankAccount::new(...)`. Rust has no constructor keyword; `new` is a naming

convention. The struct literal `BankAccount { id: ..., owner: ..., balance_cents: 0 }` builds the value.

- **The three kinds of `self`** are chapter 6's rules applied to methods: `&self` borrows the object to read it (`balance`), `&mut self` borrows it to change it (`deposit`), and plain `self` takes ownership, consuming the object. Python's `self` is always the first kind and the second at once.
- **Errors are return values.** `deposit` returns `Result<(), String>`: either `Ok(())` (success, no value; `()` is the empty tuple, Rust's "nothing") or `Err(message)`. Chapter 9 covers this fully.
- `#[derive(Debug)]` is an **attribute** that generates code: here, the ability to print the struct with `{:?}`.

7.3 Using It

src/ch07_structs.rs

```
// in run():
let mut account = BankAccount::new("ACC-0001", "Ada");
account.deposit(120_00).unwrap();
println!("{}", account.summary());

match account.withdraw(500_00) {
    Ok(()) => println!("withdrew 500.00"),
    Err(message) => println!("Refused: {message}"),
}
if let Err(message) = account.deposit(-5) {
    println!("Refused: {message}");
}
account.withdraw(20_00).unwrap();
println!("{} now has {}", account.owner(), dollars(account.balance()));
println!("{account:?}");
```

Output

```
ACC-0001 Ada: 120.00
Refused: insufficient funds: balance 120.00, requested 500.00
Refused: deposit must be positive, got -5
Ada now has 100.00
BankAccount { id: "ACC-0001", owner: "Ada", balance_cents: 10000 }
```

- `let mut account`: the variable must be `mut`, because `deposit` takes `&mut self`.
- `.unwrap()` takes the success value out of a `Result` and stops the program if it was an error. Fine in demos and tests, where an error means a bug.

- `match` handles both outcomes; `if let Err(message) = ...` handles only the error. Both are chapter 8's pattern matching.

7.4 Data Structs and derive

src/ch07_structs.rs

```rust
/// A plain data struct. derive generates code for printing ({:?}),
/// copying and comparing with ==, as @dataclass(frozen=True) does.
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct Point {
    pub x: f64,
    pub y: f64,
}

impl Point {
    pub fn distance_to(&self, other: &Point) -> f64 {
        ((self.x - other.x).powi(2) + (self.y - other.y).powi(2)).sqrt()
    }
}

/// A tuple struct: a named wrapper around one value (a "newtype").
pub struct Celsius(pub f64);

impl Celsius {
    pub fn to_fahrenheit(&self) -> f64 {
        self.0 * 9.0 / 5.0 + 32.0
    }
}
```

Output

```
Point { x: 0.0, y: 0.0 } to Point { x: 3.0, y: 4.0 } is 5
equal by value: true
37 C is 98.6 F
```

`#[derive(...)]` asks the compiler to write standard behaviour for you, as `@dataclass` writes `__init__`, `__repr__` and `__eq__`.

Derive	Gives	Python
`Debug`	Printing with `{:?}`	`__repr__`
`Clone`	`.clone()`, an explicit deep copy	`copy.deepcopy`
`Copy`	Implicit copy on assignment (small types only)	(none)
`PartialEq`	`==` and `!=`, comparing fields	`__eq__`
`Default`	`Type::default()` with zero/empty fields	default field values
`Hash, Eq`	Use as a `HashMap` key or in a `HashSet`	`__hash__`

`Celsius(pub f64)` is a **tuple struct**: fields have positions, not names (`body.0`). A one-field tuple struct is called a **newtype**. It costs nothing at run time and stops you passing a Fahrenheit value where Celsius is expected.

8 Enums and Pattern Matching

A Rust **enum** is a type whose value is exactly one of a fixed list of **variants**, and each variant can carry its own data. Python's `enum.Enum` covers only the simplest case. Together with `match`, enums are how Rust code models "one of these things". File: `src/ch08_enums.rs`.

8.1 Variants with Data

src/ch08_enums.rs

```rust
/// An enum lists every form a value can take. Each variant can carry data.
#[derive(Debug)]
pub enum Shape {
    Circle { radius: f64 },
    Rectangle { width: f64, height: f64 },
    Square(f64),
}

impl Shape {
    pub fn area(&self) -> f64 {
        // match must handle every variant, or the build fails
        match self {
            Shape::Circle { radius } => PI * radius * radius,
            Shape::Rectangle { width, height } => width * height,
            Shape::Square(side) => side * side,
        }
    }

    pub fn name(&self) -> &str {
        match self {
            Shape::Circle { .. } => "Circle",
            Shape::Rectangle { .. } => "Rectangle",
            Shape::Square(_) => "Square",
        }
    }
}
```

Each variant can be shaped differently: `Circle` has a named field, `Rectangle` two, `Square` one unnamed value. In `match self`, each arm's pattern names the variant and binds its data to variables (`radius`, `width`, `height`, `side`). `{ .. }` and `(_)` ignore the data.

src/ch08_enums.rs

```
// in run():
let shapes = vec![
    Shape::Circle { radius: 1.0 },
    Shape::Rectangle {
        width: 2.0,
        height: 3.0,
    },
    Shape::Square(2.0),
];
let mut total = 0.0;
for shape in &shapes {
    println!("{} with area {:.2}", shape.name(), shape.area());
    total += shape.area();
}
println!("Total area: {total:.2}");
```

Output

```
Circle with area 3.14
Rectangle with area 6.00
Square with area 4.00
Total area: 13.14
```

In Python you would write a `Shape` base class with three subclasses, or a dataclass per shape and an `isinstance` chain. The Rust version has a guarantee neither has: if you add a `Triangle` variant, every `match` that does not handle it becomes a compile error. Drill 5 shows this. Chapter 11 shows the other approach, a trait, which suits a list of types that other code may extend.

8.2 Simple Variants

src/ch08_enums.rs

```
/// Variants with no data work like Python's enum.Enum.
#[derive(Debug, Clone, Copy, PartialEq)]
pub enum Light {
    Red,
    Yellow,
    Green,
}

impl Light {
    pub fn next(self) -> Light {
        match self {
            Light::Red => Light::Green,
            Light::Green => Light::Yellow,
            Light::Yellow => Light::Red,
        }
    }
}
```

Output

```
Red -> Green -> Yellow -> Red
```

An enum with no data is Python's `enum.Enum`. `next` takes `self` by value, which is fine because the derived `Copy` makes the enum copy instead of move.

8.3 Patterns

src/ch08_enums.rs

```
// Patterns can test shapes, bind names and add conditions ("guards")
fn describe(point: (i32, i32)) -> String {
    match point {
        (0, 0) => "the origin".to_string(),
        (x, 0) => format!("on the x axis at {x}"),
        (0, y) => format!("on the y axis at {y}"),
        (x, y) if x == y => format!("on the diagonal at {x}"),
        (x, y) => format!("at ({x}, {y})"),
    }
}
```

Output

```
(0, 0) is the origin
(5, 0) is on the x axis at 5
(0, -2) is on the y axis at -2
(3, 3) is on the diagonal at 3
(1, 2) is at (1, 2)
```

Pattern	Matches
`0, "text", 'c'`	That exact value
`x`	Anything, and binds it to the name `x`
`_`	Anything, without binding
`1..=9`	A value in the inclusive range
`'a' | 'e' | 'i'`	Any of the alternatives
`(x, 0)`	A tuple whose second item is 0
`Shape::Square(side)`	That variant, binding its data
`x if x > 5`	A **match guard**: the pattern, plus a condition

8.4 Option Is an Enum

`Option<T>` from section 4.4 is defined in the standard library as an ordinary enum: `enum Option<T> { Some(T), None }`. So you take it apart with `match`.

src/ch08_enums.rs

```
// Option<T> is an ordinary enum: Some(value) or None
fn lookup(id: u32) -> Option<String> {
    match id {
        1 => Some("Ada".to_string()),
        2 => Some("Grace".to_string()),
        _ => None,
    }
}

// in run():
for id in [1, 3] {
    match lookup(id) {
        Some(name) => println!("user {id} is {name}"),
        None => println!("user {id} not found"),
    }
}
// if let: match one pattern and ignore the rest
if let Some(name) = lookup(2) {
    println!("found {name} with if let");
}
```

Output

```
user 1 is Ada
user 3 not found
found Grace with if let
```

`if let PATTERN = value { ... }` is a `match` with one arm that you care about; the other cases do nothing. Python's nearest code is `if (name := lookup(2)) is not None:`, but the Rust version cannot forget the check, because the name exists only inside the block where the `Some` matched.

9 Error Handling: Result and Option

Rust has no exceptions. A function that can fail returns `Result<T, E>`, an enum with two variants: `Ok(T)` holding the value, or `Err(E)` holding the error. The failure is part of the function's type, so callers cannot forget it exists. File: `src/ch09_errors.rs`.

9.1 Handling a Result

src/ch09_errors.rs

```
// The long way: match on the Result that parse returns
fn parse_age(text: &str) -> Result<u32, String> {
    match text.trim().parse::<u32>() {
        Ok(age) => Ok(age),
        Err(e) => Err(format!("'{text}' is not an age: {e}")),
    }
}
```

The Python equivalent:

Python

```
def parse_age(text: str) -> int:
    try:
        return int(text.strip())
    except ValueError as e:
        raise ValueError(f"'{text}' is not an age: {e}")
```

`parse::<u32>()` converts text to a number and returns `Result<u32, ParseIntError>`. The `::<u32>` ("turbofish") tells it which number type to produce. The `match` handles both variants; returning `Err(...)` is the Rust form of `raise`.

9.2 The ? Operator

src/ch09_errors.rs

```
// The short way: `?` returns the error to the caller, or unwraps the value
fn sum_numbers(text: &str) -> Result<i64, ParseIntError> {
    let mut total = 0;
    for part in text.split(',') {
        total += part.trim().parse::<i64>()?;
    }
    Ok(total)
}
```

Output

```
Ok(6)
could not sum: invalid digit found in string
```

`expr?` means: if `expr` is `Ok(v)`, the expression's value is `v`; if it is `Err(e)`, return `Err(e)` from this function now. It is how an error travels up the call stack, the job an uncaught exception does in Python, but every place it can happen is marked with a visible `?`. It works only inside a function that itself returns a `Result` (or an `Option`, where it returns `None` early). When the error types differ, `?` converts the error with the `From` trait if a conversion exists.

9.3 A Custom Error Type

Real code defines its own error enum, one variant per way the operation can fail. A caller can then `match` on the variant, which is Python's `except` with several exception classes.

src/ch09_errors.rs

```rust
/// A custom error type: one enum variant per way the operation can fail.
#[derive(Debug)]
pub enum ConfigError {
    Missing(String),
    NotANumber { key: String, value: String },
    OutOfRange { key: String, value: i64 },
}

// Display gives the error its human-readable message (Python: __str__)
impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            ConfigError::Missing(key) => write!(f, "setting '{key}' is missing"),
            ConfigError::NotANumber { key, value } => {
                write!(f, "setting '{key}' should be a number, got '{value}'")
            }
            ConfigError::OutOfRange { key, value } => {
                write!(f, "setting '{key}' is out of range: {value}")
            }
        }
    }
}

// Marks ConfigError as an error type that other error-handling code accepts
impl std::error::Error for ConfigError {}
```

src/ch09_errors.rs

```rust
// Find `key` in text made of "key=value" lines.
fn find_setting(text: &str, key: &str) -> Option<String> {
    for line in text.lines() {
        // if let ... && ...: the pattern must match and the condition hold
```

```
        if let Some((k, v)) = line.split_once('=')
            && k.trim() == key
        {
            return Some(v.trim().to_string());
        }
    }
    None
}

/// Read a TCP port from config text, explaining what is wrong if it can't.
pub fn read_port(config: &str) -> Result<u16, ConfigError> {
    // ok_or turns an Option into a Result: None becomes the given error
    let raw = find_setting(config,
"port").ok_or(ConfigError::Missing("port".to_string()))?;
    let value: i64 = match raw.parse() {
        Ok(n) => n,
        Err(_) => {
            return Err(ConfigError::NotANumber {
                key: "port".to_string(),
                value: raw,
            });
        }
    };
    if !(1..=65535).contains(&value) {
        return Err(ConfigError::OutOfRange {
            key: "port".to_string(),
            value,
        });
    }
    Ok(value as u16)
}
```

Output

```
port 5432
config error: setting 'port' is missing
config error: setting 'port' should be a number, got 'http'
config error: setting 'port' is out of range: 70000
```

- `impl fmt::Display for ConfigError` gives each variant its message, so `{e}` prints it. Python's equivalent is `__str__` on an exception class.
- `impl std::error::Error for ConfigError {}` marks it as an error type; the trait's methods all have defaults, so the block is empty. Chapter 11 explains traits.
- `ok_or(...)` turns an `Option` into a `Result`: `None` becomes the given error, which `?` then returns.
- `if let Some((k, v)) = ... && k.trim() == key` needs the 2024 edition. In older code you will see two nested `if`s instead.

- Libraries named `thiserror` (for defining error enums) and `anyhow` (for applications that only report errors) remove most of this boilerplate. Interviewers often mention them.

9.4 Panics and Shortcuts

src/ch09_errors.rs

```
// in run():
// Shortcuts. unwrap and expect panic (crash) on an error, so they belong
// in tests and quick scripts, not in code that should keep running.
let port = read_port("port=8080").expect("config should be valid");
let fallback = read_port("").unwrap_or(3000);
println!("port {port}, fallback {fallback}");

// A panic is for bugs, not expected failures. Indexing past the end panics;
// .get returns an Option instead.
let items = [10, 20, 30];
println!("get(1) = {:?}, get(9) = {:?}", items.get(1), items.get(9));
```

Output

```
port 8080, fallback 3000
get(1) = Some(20), get(9) = None
```

A **panic** stops the current thread with a message, and usually the whole program. It is for bugs, situations the code does not expect to handle: indexing past the end of an array, `unwrap` on an `Err`, an explicit `panic!("...")`. `Result` is for failures you expect: bad input, a missing file, a network error. The rule of thumb is "recoverable: `Result`; bug: panic".

Method on Result or Option	Does
`unwrap()`	The value, or panic
`expect("msg")`	The value, or panic with your message. Preferred over `unwrap`.
`unwrap_or(default)`	The value, or the default
`map(f)`	Apply `f` to the success value, leave an error alone
`map_err(f)`	Convert the error, leave a success alone
`ok_or(err)`	`Option` to `Result`
`ok()`	`Result` to `Option`, dropping the error
`is_ok(), is_err(), is_some(), is_none()`	Test without taking apart

10 Collections

The standard library has the collections you use in Python, under different names. All of them hold one element type, written in angle brackets: `Vec<String>` is a list of strings. File: `src/ch10_collections.rs`.

Python	Rust	Notes
`list`	`Vec<T>`	Growable, ordered. `vec![1, 2, 3]` builds one.
`dict`	`HashMap<K, V>`	Iteration order is random, run to run.
`dict, sorted keys`	`BTreeMap<K, V>`	Iterates in key order.
`set`	`HashSet<T>`	Also `BTreeSet`, sorted.
`collections.deque`	`VecDeque<T>`	Fast at both ends.
`heapq`	`BinaryHeap<T>`	A max-heap.
`tuple`	`(A, B)`	Built into the language.

10.1 Vec and Slices

src/ch10_collections.rs

```
// in run():
// Vec<T>: a growable list (Python: list). vec! builds one from values.
let mut languages = vec!["Python", "Rust", "Go"];
languages.push("SQL");
languages.sort();
println!("{languages:?} has {} items", languages.len());
println!(
    "first {:?}, tenth {:?}",
    languages.first(),
    languages.get(9)
);
println!("contains Rust? {}", languages.contains(&"Rust"));

// A slice borrows part of a Vec (Python: languages[1:3], but no copy)
let middle: &[&str] = &languages[1..3];
println!("middle {middle:?}");
```

Output

```
["Go", "Python", "Rust", "SQL"] has 4 items
first Some("Go"), tenth None
contains Rust? true
middle ["Python", "Rust"]
```

`first()` and `get(9)` return an `Option`, so an empty list or a bad index is a `None` to handle, not an `IndexError`. `&languages[1..3]` is a **slice**: a borrowed view of part of the vector, with no copy. A function that only reads a list should take `&[T]`, which accepts a `Vec`, an array or a slice.

10.2 HashMap and BTreeMap

src/ch10_collections.rs

```rust
// in run():
// HashMap<K, V>: Python's dict
let mut stock: HashMap<String, u32> = HashMap::new();
stock.insert("apple".to_string(), 3);
stock.insert("pear".to_string(), 5);
// entry().or_insert() is Python's dict.setdefault
*stock.entry("apple".to_string()).or_insert(0) += 2;
println!("apples: {}", stock["apple"]);
match stock.get("plum") {
    Some(count) => println!("plums: {count}"),
    None => println!("no plums"),
}

// HashMap iteration order is random, run to run. BTreeMap keeps keys
// sorted, so use it (or sort the keys) whenever the order is shown.
let text = "the cat sat on the mat and the dog sat too";
let mut counts: BTreeMap<&str, u32> = BTreeMap::new();
for word in text.split_whitespace() {
    *counts.entry(word).or_insert(0) += 1;
}
println!("{counts:?}");
```

Output

```
apples: 5
no plums
{"and": 1, "cat": 1, "dog": 1, "mat": 1, "on": 1, "sat": 2, "the": 3, "too": 1}
```

- `stock["apple"]` panics if the key is missing, like a `KeyError`. `stock.get("plum")` returns an `Option`, like `dict.get`.
- `entry(key).or_insert(0)` returns a mutable reference to the value, inserting 0 first if the key is new. `*... += 1` adds to the value through the reference (`*` follows the reference). It is the word-count idiom; Python writes `counts[w] = counts.get(w, 0) + 1` or uses `Counter`.
- Python's `dict` remembers insertion order. `HashMap` does not, and its order changes from run to run. Use `BTreeMap`, or sort, whenever order is visible.

10.3 HashSet and Iterating

src/ch10_collections.rs

```
// in run():
// HashSet<T>: Python's set
let rust_devs = HashSet::from(["Ada", "Grace", "Linus"]);
let python_devs = HashSet::from(["Grace", "Guido", "Linus"]);
// copied() turns the &&str items intersection yields into plain &str
let mut both: Vec<&str> = rust_devs.intersection(&python_devs).copied().collect();
both.sort();
println!("know both: {both:?}");
println!("Guido knows Rust? {}", rust_devs.contains("Guido"));

// Iterating: & borrows each item; without & the loop consumes the Vec
let mut prices = vec![100, 250, 75];
for price in &mut prices {
    *price += 10; // * reaches through the reference to change the item
}
for price in &prices {
    print!("{price} ");
}
println!();
```

Output

```
know both: ["Grace", "Linus"]
Guido knows Rust? false
110 260 85
```

The three ways to loop over a collection are chapter 6 again:

Loop	Each item is	The collection afterwards
`for x in &v`	A shared reference, `&T`	Unchanged and usable
`for x in &mut v`	A mutable reference, `&mut T`	Possibly changed, usable
`for x in v`	The value itself, `T` (moved out)	Gone: consumed by the loop

11 Traits and Generics

A **trait** is a named set of methods that a type promises to provide. It is Rust's answer to interfaces, abstract base classes and Python's `Protocol`. Rust has no inheritance between structs: shared behaviour comes from traits, and shared data from putting one struct inside another (composition). File: `src/ch11_traits.rs`.

11.1 Defining and Implementing a Trait

src/ch11_traits.rs

```rust
/// A trait lists methods a type must provide (Python: an ABC or a Protocol).
pub trait Describe {
    fn name(&self) -> String;

    // A default method: implementing types get it free and may replace it
    fn describe(&self) -> String {
        format!("This is {}", self.name())
    }
}

pub struct Dog {
    pub name: String,
}

pub struct Robot {
    pub model: u32,
}

impl Describe for Dog {
    fn name(&self) -> String {
        self.name.clone()
    }

    fn describe(&self) -> String {
        format!("{} says woof", self.name)
    }
}

impl Describe for Robot {
    fn name(&self) -> String {
        format!("Robot #{}", self.model)
    }
}
```

- `fn name(&self) -> String;` ends in a semicolon: it has no body, so every implementing type must write one. Python: an `@abstractmethod`.
- `describe` has a body, a **default method**. `Dog` replaces it; `Robot` keeps it.

- `impl Describe for Dog` is written separately from the struct. You can implement your own trait for someone else's type, or someone else's trait for your type, but not someone else's trait for someone else's type (the **orphan rule**).
- Python's `Protocol` is matched by shape: any class with a `name` method fits. A Rust type fits a trait only with an explicit `impl`.

11.2 Generics: Static Dispatch

src/ch11_traits.rs

```
// Generic function: T is any type that implements Describe. The compiler
// writes a separate copy for each T used ("static dispatch").
pub fn announce<T: Describe>(item: &T) -> String {
    format!("Announcing: {}", item.describe())
}

// Generic with two trait bounds: works for i32, f64, char, ...
pub fn largest<T: PartialOrd + Copy>(items: &[T]) -> T {
    let mut best = items[0];
    for &item in items {
        if item > best {
            best = item;
        }
    }
    best
}
```

`<T: Describe>` declares a **type parameter** `T` with a **trait bound**: `T` may be any type that implements `Describe`. The compiler checks the bound where the function is called, and generates a separate copy of `announce` for each type it is used with (**monomorphisation**). The call is then as fast as a direct call. `largest` has two bounds joined by `+`: `PartialOrd` allows `>`, `Copy` allows `let mut best = items[0]` without moving out of the slice.

src/ch11_traits.rs

```
// in run():
let rex = Dog {
    name: "Rex".to_string(),
};
let r2 = Robot { model: 2 };
println!("{}", announce(&rex));
println!("{}", announce(&r2));

let biggest_int = largest(&[3, 9, 4]);
let biggest_float = largest(&[1.5, 0.2]);
let biggest_char = largest(&['x', 'b']);
println!("largest: {biggest_int} {biggest_float} {biggest_char}");
```

Output

```
Announcing: Rex says woof
Announcing: This is Robot #2
largest: 9 1.5 x
```

Python's `TypeVar("T", bound=...)` expresses the same idea for a type checker; Rust's compiler enforces it. The short form `fn announce(item: &impl Describe)` means the same as the generic version.

11.3 Trait Objects: Dynamic Dispatch

src/ch11_traits.rs

```
// in run():
// A trait object (Box<dyn Describe>) lets one Vec hold different types.
// The method to call is looked up at run time ("dynamic dispatch").
let things: Vec<Box<dyn Describe>> = vec![
    Box::new(Dog {
        name: "Fido".to_string(),
    }),
    Box::new(Robot { model: 7 }),
];
for thing in &things {
    println!("{}", thing.describe());
}
```

Output

```
Fido says woof
This is Robot #7
```

A `Vec` holds one type. To hold a `Dog` and a `Robot` together, store each as a `Box<dyn Describe>`: a pointer to a value on the heap whose concrete type is known only at run time, plus a table of its methods (a **vtable**). Calling `describe` looks up the method in the table. This is how every Python method call works, and how C# and Java virtual methods work.

	Generic `<T: Trait>`	Trait object `dyn Trait`
Method chosen	At compile time	At run time, through a vtable
Speed	Fastest; can be inlined	One extra pointer lookup per call
Code size	One copy per type used	One copy

	Generic `<T: Trait>`	Trait object `dyn Trait`
Mixed types in one collection	No	Yes
Typical use	Most library code; `OrderService<D>` in chapter 15	Plugin lists, `Box<dyn Error>`

11.4 Standard Traits: Display and Operators

Implementing standard-library traits gives your type built-in behaviour, the way Python's dunder methods do.

src/ch11_traits.rs

```
// Display: what {} prints (Python: __str__)
impl fmt::Display for Fraction {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}/{}", self.num, self.den)
    }
}

// Add: what + does (Python: __add__)
impl Add for Fraction {
    type Output = Fraction;

    fn add(self, other: Fraction) -> Fraction {
        Fraction::new(
            self.num * other.den + other.num * self.den,
            self.den * other.den,
        )
    }
}
```

src/ch11_traits.rs

```
// in run():
let half = Fraction::new(1, 2);
let third = Fraction::new(1, 3);
println!("{half} + {third} = {}", half + third);
println!("debug: {:?}", half + half);
println!("2/4 == 1/2? {}", Fraction::new(2, 4) == half);
```

Output

```
1/2 + 1/3 = 5/6
debug: Fraction { num: 1, den: 1 }
2/4 == 1/2? true
```

Trait	Enables	Python
`Display`	`{}` formatting and `.to_string()`	`__str__`
`Debug`	`{:?}` formatting	`__repr__`
`Add, Sub, Mul, ...`	`+`, `-`, `*`, ...	`__add__, ...`
`PartialEq, PartialOrd`	`==`, `<`, ...	`__eq__, __lt__`
`Iterator`	`for` loops and iterator methods	`__iter__, __next__`
`Drop`	Code that runs when a value is freed	`__exit__, __del__`
`From / Into`	Conversions; used by `?`	a constructor classmethod
`Default`	`Type::default()`	default arguments

`type Output = Fraction;` inside `impl Add` is an **associated type**: the trait lets each implementation choose the type that `+` returns.

12 Iterators and Closures

Where Python uses comprehensions and generator expressions, Rust chains iterator methods, each taking a small anonymous function called a **closure**. File: `src/ch12_iterators.rs`.

12.1 Closures

src/ch12_iterators.rs

```
// in run():
// A closure is an anonymous function (Python: lambda). It can use
// variables from the surrounding code.
let bonus = 500;
let with_bonus = |salary: u32| salary + bonus;
println!("with bonus: {}", with_bonus(1_000));
```

Output

```
with bonus: 1500
```

`|salary: u32| salary + bonus` is Python's `lambda salary: salary + bonus`. Parameters go between the bars; types are usually inferred. A closure body can be a whole block in braces, unlike a Python lambda. It can use (**capture**) variables from where it is written, here `bonus`. Writing `move` before the bars makes the closure take ownership of what it captures, which chapter 14 needs for threads.

12.2 Filter, Map, Collect

The data is a list of `Employee` structs:

src/ch12_iterators.rs

```rust
/// One member of staff.
#[derive(Debug, Clone)]
pub struct Employee {
    pub name: String,
    pub dept: String,
    pub salary: u32,
    pub years: u32,
}

pub fn staff() -> Vec<Employee> {
    vec![
        Employee::new("Ada", "Engineering", 120_000, 5),
        Employee::new("Grace", "Engineering", 135_000, 9),
        Employee::new("Linus", "Engineering", 98_000, 2),
        Employee::new("Margaret", "Sales", 87_000, 4),
        Employee::new("Alan", "Sales", 91_000, 0),
        Employee::new("Barbara", "Finance", 105_000, 7),
    ]
}
```

src/ch12_iterators.rs

```rust
// in run():
// Python: [e.name for e in staff if e.salary > 100_000]
let high_earners: Vec<&str> = staff
    .iter()
    .filter(|e| e.salary > 100_000)
    .map(|e| e.name.as_str())
    .collect();
println!("High earners: {}", high_earners.join(", "));
```

Output

```
High earners: Ada, Grace, Barbara
```

1. `staff.iter()` makes an iterator that yields a reference, `&Employee`, to each item.
2. `.filter(|e| ...)` keeps items for which the closure returns `true`.
3. `.map(|e| ...)` turns each item into something else.
4. `.collect()` runs the chain and gathers the results into a collection. Which collection depends on the type you ask for: here `Vec<&str>`. It could equally be a `HashSet` or, from pairs, a `HashMap`.

12.3 Aggregates and Sorting

src/ch12_iterators.rs

```
// in run():
// Aggregates
let payroll: u32 = staff.iter().map(|e| e.salary).sum();
let average_years = staff.iter().map(|e| e.years).sum::<u32>() as f64 /
staff.len() as f64;
let any_new = staff.iter().any(|e| e.years == 0);
let in_hr = staff.iter().find(|e| e.dept == "HR");
println!("Payroll {payroll}, avg years {average_years:.1}, new hires? {any_new}");
println!("HR: {:?}", in_hr.map(|e| &e.name));

// Python: sorted(staff, key=lambda e: (-e.years, e.name))
let mut by_seniority = staff.clone();
by_seniority.sort_by(|a, b| b.years.cmp(&a.years).then(a.name.cmp(&b.name)));
println!("Most senior: {}", by_seniority[0].name);

let max_paid = staff.iter().max_by_key(|e| e.salary).unwrap();
println!("Top salary: {} ({})", max_paid.name, max_paid.salary);
```

Output

```
Payroll 636000, avg years 4.5, new hires? true
HR: None
Most senior: Grace
Top salary: Grace (135000)
```

Python	Rust
`[f(x) for x in xs]`	`xs.iter().map(f).collect()`
`[x for x in xs if p(x)]`	`xs.iter().filter(p).collect()`
`sum(x.n for x in xs)`	`xs.iter().map(|x| x.n).sum()`
`any(p(x) for x in xs)`	`xs.iter().any(p)`
`all(p(x) for x in xs)`	`xs.iter().all(p)`
`next((x for x in xs if p(x)), None)`	`xs.iter().find(p)`
`max(xs, key=k)`	`xs.iter().max_by_key(k)`
`len([x for x in xs if p(x)])`	`xs.iter().filter(p).count()`
`xs.sort(key=k)`	`xs.sort_by_key(k)`
`zip(a, b), enumerate(xs)`	`a.iter().zip(b.iter()), xs.iter().enumerate()`
`reversed(xs)`	`xs.iter().rev()`
`itertools.islice(xs, 4)`	`xs.take(4)`

`sort_by` takes a closure that compares two items and returns an `Ordering`. `b.years.cmp(&a.years)` compares in reverse, for most years first, and `.then(...)` breaks ties by name. Rust's sort is stable, like Python's. `staff.clone()` copies the list so the sort does not reorder `staff`.

12.4 Laziness

src/ch12_iterators.rs

```
// in run():
// Iterators are lazy: nothing runs until something consumes them.
// Python generators behave the same way.
let squares = (1..).map(|n| n * n).filter(|n| n % 2 == 1);
let first_four: Vec<u64> = squares.take(4).collect();
println!("first four odd squares: {first_four:?}");
```

Output

```
first four odd squares: [1, 9, 25, 49]
```

`(1..)` is an endless range, and `map` and `filter` do no work when called; they build a description of the work. Only `take(4).collect()` pulls values through, and it stops after four. Python generators behave the same way. Rust iterator chains compile down to the same machine code as a hand-written loop, an example of what Rust calls **zero-cost abstractions**.

12.5 iter, iter_mut, into_iter

Method	Yields	Same as the loop
`v.iter()`	`&T`, shared references	`for x in &v`
`v.iter_mut()`	`&mut T`, mutable references	`for x in &mut v`
`v.into_iter()`	`T`, owned values; `v` is consumed	`for x in v`

13 Lifetimes and Smart Pointers

This chapter finishes the ownership story: how the compiler tracks how long a reference is valid, what happens when a value is dropped, and the types to reach for when one owner is not enough. File: `src/ch13_lifetimes.rs`.

13.1 Lifetimes

A **lifetime** is the span of code during which a reference is valid. The compiler checks that no reference outlives the value it points to (the second half of section 6.4's rule). Usually it works the lifetimes out alone. When a function returns a reference and takes more than one, it cannot tell which input the result borrows from, and you must say:

src/ch13_lifetimes.rs

```
// A lifetime ('a) tells the compiler how long a borrowed value must live.
// This says: the result borrows from x or y, so it is valid only while
// both of them are.
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

/// A struct that holds a reference needs a lifetime too: a Headline
/// may not outlive the text it points into.
pub struct Headline<'a> {
    pub text: &'a str,
}

impl<'a> Headline<'a> {
    pub fn first_word(&self) -> &'a str {
        self.text.split_whitespace().next().unwrap_or("")
    }
}
```

- `'a` (said "tick a" or "lifetime a") is a lifetime parameter, declared in angle brackets like a type parameter. `longest<'a>(x: &'a str, y: &'a str) -> &'a str` says: the result is valid as long as both inputs are.
- Lifetime annotations do not change how long anything lives. They describe relationships so the compiler can check the callers.
- A struct that holds a reference needs a lifetime parameter too: a `Headline` may not outlive the text it points into. Most structs avoid this by owning their data (`String`, not `&str`).
- `'static` is the lifetime of the whole program. String literals are `&'static str`.

src/ch13_lifetimes.rs

```
// in run():
let a = String::from("a short line");
let result;
{
    let b = String::from("a much longer line");
    result = longest(&a, &b);
    println!("longest: {result}");
} // b is dropped here, so `result` may not be used after this brace

let article = String::from("Rust reaches version two");
let headline = Headline { text: &article };
println!("first word: {}", headline.first_word());
```

Output

```
longest: a much longer line
first word: Rust
```

Moving the `println!("longest: ...")` line below the inner block's closing brace is a compile error: `b` is dropped at that brace, and `result` might point into it.

13.2 Drop: Cleanup at Scope End

src/ch13_lifetimes.rs

```
/// Drop runs when a value goes out of scope, like a `with` block's
/// __exit__. Files, locks and sockets close themselves this way (RAII).
pub struct Resource {
    pub name: String,
}

impl Drop for Resource {
    fn drop(&mut self) {
        println!("  closing {}", self.name);
    }
}
```

src/ch13_lifetimes.rs

```
// in run():
println!("before the block");
{
    let _file = Resource {
        name: "data.csv".to_string(),
    };
    let _conn = Resource {
        name: "database".to_string(),
    };
    println!("  using both");
} // dropped in reverse order of creation
println!("after the block");
```

Output

```
before the block
  using both
  closing database
  closing data.csv
after the block
```

The `drop` method of the `Drop` trait runs automatically when a value's owner goes out of scope, in reverse order of creation. Rust's `File`, `MutexGuard` and network connections use it to close or release themselves, so Rust needs no `with` statement and no `finally`. The pattern is called **RAII** ("resource acquisition is initialisation"), a name taken from C++.

13.3 Box: Values on the Heap

src/ch13_lifetimes.rs

```
/// Box<T> puts a value on the heap. A recursive type needs it, because
/// otherwise its size would be infinite.
pub enum Expr {
    Number(i64),
    Add(Box<Expr>, Box<Expr>),
    Mul(Box<Expr>, Box<Expr>),
}

pub fn eval(expr: &Expr) -> i64 {
    match expr {
        Expr::Number(n) => *n,
        Expr::Add(a, b) => eval(a) + eval(b),
        Expr::Mul(a, b) => eval(a) * eval(b),
    }
}
```

src/ch13_lifetimes.rs

```
// in run():
// (2 + 3) * 4 as a tree
let expr = Expr::Mul(
    Box::new(Expr::Add(
        Box::new(Expr::Number(2)),
        Box::new(Expr::Number(3)),
    )),
    Box::new(Expr::Number(4)),
);
println!("(2 + 3) * 4 = {}", eval(&expr));
```

Output

```
(2 + 3) * 4 = 20
```

Every Rust type must have a size known at compile time. An `Expr` that contained an `Expr` directly would be infinitely large. `Box<Expr>` is a pointer to an `Expr` on the heap, so its size is fixed. `Box` has one owner and frees its contents when dropped. It is also how you store a trait object (`Box<dyn Describe>`, section 11.3).

13.4 Rc and RefCell: Shared Ownership

src/ch13_lifetimes.rs

```
// in run():
// Rc<T>: several owners of one value, freed when the last owner goes.
// RefCell<T>: allows changes through a shared reference, checked at run time.
// Together they give Python-style shared, mutable objects.
let shared = Rc::new(RefCell::new(vec![1, 2]));
let other_owner = Rc::clone(&shared);
other_owner.borrow_mut().push(3);
println!(
    "shared = {:?}, owners = {}",
    shared.borrow(),
    Rc::strong_count(&shared)
);
```

Output

```
shared = [1, 2, 3], owners = 2
```

Sometimes one owner is not enough: a graph node with several parents, or a cache shared by several parts of a program. `Rc<T>` (reference counted) allows several owners. `Rc::clone` copies the pointer, not the data, and adds one to a counter; the value is freed when the counter reaches zero. This is exactly how CPython manages memory.

`Rc` gives only shared (read-only) access. `RefCell<T>` allows changing the value through a shared reference by moving the borrow check from compile time to run time: `borrow_mut()` panics if the value is already borrowed. This is called **interior mutability**.

Type	Use when
`Box<T>`	One owner, value on the heap: recursive types, trait objects
`Rc<T>`	Several owners, one thread
`Arc<T>`	Several owners across threads (atomic counter; chapter 14)
`RefCell<T>`	Change through a shared reference, one thread; checked at run time
`Mutex<T>`	Change through a shared reference across threads; one at a time

Common pairings are `Rc<RefCell<T>>` in single-threaded code and `Arc<Mutex<T>>` in multi-threaded code. Two `Rc`s that point at each other are never freed (a leak); `Weak` pointers break such cycles.

14 Concurrency and Async

Rust's ownership rules also prevent **data races**: two threads touching the same data at once with at least one writing. A program with a possible data race does not compile. Rust calls this "fearless concurrency". File: `src/ch14_concurrency.rs`.

14.1 Threads

src/ch14_concurrency.rs

```
// in run():
// 1. Threads run in parallel on separate CPU cores; there is no GIL.
//    `move` gives each thread its own copy of `chunk`.
let numbers: Vec<u64> = (1..=1_000).collect();
let mut handles = Vec::new();
for chunk in numbers.chunks(250) {
    let chunk = chunk.to_vec();
    handles.push(thread::spawn(move || chunk.iter().sum::<u64>()));
}
let mut total = 0;
for handle in handles {
    total += handle.join().unwrap(); // join waits for the thread to finish
}
println!("sum of 1..=1000 over 4 threads: {total}");
```

Output

```
sum of 1..=1000 over 4 threads: 500500
```

- `thread::spawn` starts an operating-system thread running the closure and returns a `JoinHandle`. `join()` waits for it and returns the closure's result. Python: `threading.Thread`, or `concurrent.futures`.
- There is no GIL. The four threads run on four cores at once, which is why CPU-heavy Python work moves to Rust.
- `move` makes the closure take ownership of `chunk`. Without it the closure would borrow a local variable that might be gone before the thread finishes, and the compiler refuses.

14.2 Shared State: Arc and Mutex

src/ch14_concurrency.rs

```rust
// in run():
// 2. Shared, changing state: Arc (shared ownership across threads)
//    around a Mutex (one thread at a time).
let counter = Arc::new(Mutex::new(0));
let mut handles = Vec::new();
for _ in 0..8 {
    let counter = Arc::clone(&counter);
    handles.push(thread::spawn(move || {
        for _ in 0..1_000 {
            *counter.lock().unwrap() += 1;
        }
    }));
}
for handle in handles {
    handle.join().unwrap();
}
println!(
    "counter after 8 threads x 1000: {}",
    *counter.lock().unwrap()
);
```

Output

```
counter after 8 threads x 1000: 8000
```

- `Mutex<i32>` wraps the data it protects. The only way to reach the number is `lock()`, so you cannot forget to lock. Python's `threading.Lock` is separate from the data, and nothing stops code touching the data without it.
- `lock()` returns a guard. The lock is released when the guard is dropped, here at the end of the statement (section 13.2).
- `lock()` returns a `Result` because a thread that panicked while holding the lock leaves it "poisoned". `unwrap()` passes that panic on.
- `Arc` is the thread-safe `Rc`. Each thread gets its own `Arc` pointing at the same `Mutex`.

The compiler enforces this with two marker traits. `Send`: the type may be moved to another thread. `Sync`: the type may be shared between threads by reference. `Rc` is neither, so trying to pass an `Rc` to `thread::spawn` is a compile error; `Arc` is both.

14.3 Channels

src/ch14_concurrency.rs

```
// in run():
// 3. Channels: threads send messages instead of sharing memory
let (sender, receiver) = mpsc::channel();
for id in 1..=3 {
    let sender = sender.clone();
    thread::spawn(move || {
        sender.send(id * 10).unwrap();
    });
}
drop(sender); // close our copy so the loop below ends after the last message
let mut received: Vec<i32> = receiver.iter().collect();
received.sort();
println!("received {received:?}");
```

Output

```
received [10, 20, 30]
```

A channel sends values from threads to a receiver, like Python's `queue.Queue`. `mpsc` stands for "multiple producer, single consumer". Sending moves the value, so the sender can no longer touch it: no sharing, so no lock is needed. The receiver's loop ends when every sender has been dropped. The results are sorted because the threads may finish in any order.

14.4 Async and Await

src/ch14_concurrency.rs

```
// in run():
// 4. Async needs a runtime, which Rust does not include. Tokio is the
//    usual choice. block_on is Python's asyncio.run.
let runtime = tokio::runtime::Runtime::new().unwrap();
runtime.block_on(fetch_both());

// An async fn returns a Future: work that runs only when awaited.
async fn fetch(name: &str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    format!("{name} ({millis} ms)")
}

async fn fetch_both() {
    let start = Instant::now();
    // join! runs both futures at once (Python: asyncio.gather)
    let (a, b) = tokio::join!(fetch("prices", 200), fetch("news", 100));
    println!("got {a} and {b}");
    let waited = start.elapsed().as_millis();
    println!("waited under 300 ms, not 200 + 100: {}", waited < 300);
}
```

Output

```
got prices (200 ms) and news (100 ms)
waited under 300 ms, not 200 + 100: true
```

The syntax matches Python's: `async fn` and `.await` (written after the expression). The differences matter in an interview:

- **No built-in runtime.** Python ships `asyncio`. Rust's standard library defines futures but no event loop to run them; you choose a crate. **Tokio** is the de facto standard, and the web API in chapter 16 runs on it. `runtime.block_on(...)` is `asyncio.run(...)`; in a program whose `main` is async, the `#[tokio::main]` attribute does the same.
- **Futures are lazy.** Calling `fetch(...)` does nothing until the result is awaited or passed to something like `join!`. A Python coroutine is also lazy, but `asyncio.create_task` schedules it to run straight away.
- `tokio::join!` runs both futures concurrently and waits for both: `asyncio.gather`. `tokio::spawn` starts an independent task: `asyncio.create_task`.
- Tokio's default runtime spreads tasks across several threads, so the data a task uses must be `Send`. Python's event loop is single-threaded.

- Use async for many waits at once (network servers, many HTTP calls); use threads for CPU-heavy work. This is the same rule as in Python.

15 Walkthrough: The Order System

The order system in `src/orders/` turns an order into a receipt. It checks stock, removes the items from stock, applies a discount and adds tax. It is a small, realistic program that uses structs, an enum for errors, a trait, a generic type, a `HashMap` and iterators together. This is the code to practise changing.

15.1 Module Layout

src/orders/mod.rs

```
pub mod discounts;
pub mod models;
pub mod service;

pub use discounts::{BulkDiscount, DiscountPolicy, NoDiscount};
pub use models::{Order, OrderError, OrderLine, Product, Receipt};
pub use service::{Inventory, OrderService, TAX_RATE_PERCENT};
```

`src/orders/mod.rs` is the module's root file, as `__init__.py` is for a Python package. The `pub use` lines **re-export** the main types, so callers write `tutorial::orders::Order` rather than `tutorial::orders::models::Order`. Inside the module, `super::` means the parent module and `crate::` the root of this library.

15.2 The Data Types

src/orders/models.rs

```
/// One item for sale. Serialize lets the web API send it as JSON.
#[derive(Debug, Clone, PartialEq, Serialize)]
pub struct Product {
    pub sku: String,
    pub name: String,
    pub price_cents: i64,
}

impl Product {
    pub fn new(sku: &str, name: &str, price_cents: i64) -> Product {
        Product {
            sku: sku.to_string(),
            name: name.to_string(),
            price_cents,
        }
    }
}

/// One row of an order: which product, how many.
```

```
#[derive(Debug, Clone, PartialEq)]
pub struct OrderLine {
    pub product: Product,
    pub quantity: u32,
}

impl OrderLine {
    pub fn subtotal(&self) -> i64 {
        self.product.price_cents * self.quantity as i64
    }
}
```

src/orders/models.rs

```
/// An order being built up, line by line.
#[derive(Debug, Clone)]
pub struct Order {
    pub id: String,
    pub customer: String,
    pub lines: Vec<OrderLine>,
}

impl Order {
    pub fn new(id: &str, customer: &str) -> Order {
        Order {
            id: id.to_string(),
            customer: customer.to_string(),
            lines: Vec::new(),
        }
    }

    pub fn add(&mut self, product: Product, quantity: u32) -> Result<(),
OrderError> {
        if quantity == 0 {
            return Err(OrderError::ZeroQuantity {
                order_id: self.id.clone(),
            });
        }
        self.lines.push(OrderLine { product, quantity });
        Ok(())
    }

    pub fn subtotal(&self) -> i64 {
        self.lines.iter().map(|line| line.subtotal()).sum()
    }
}

/// What checkout returns.
#[derive(Debug, Clone, PartialEq, Serialize)]
pub struct Receipt {
    pub order_id: String,
    pub subtotal_cents: i64,
    pub discount_cents: i64,
    pub tax_cents: i64,
```

```
    pub total_cents: i64,
}
```

- Amounts are `i64` cents, and `money::dollars` formats them for display. Floats cannot hold 0.10 exactly, so money code avoids them; in Python you would use `Decimal`.
- `quantity: u32` is unsigned, so a negative quantity cannot be represented at all. `Order::add` still has to reject zero.
- An `OrderLine` owns a copy of its `Product`. Callers pass `mouse.clone()`, keeping their own. Python would share one object between both places.
- `self.lines.iter().map(|line| line.subtotal()).sum()` is Python's `sum(line.subtotal() for line in self.lines)`.
- `Serialize` comes from **serde**, the standard serialisation crate. Deriving it lets the web API send `Product` and `Receipt` as JSON (Python: Pydantic's `model_dump_json`).

15.3 Errors as an Enum

src/orders/models.rs

```
/// Every way an order can fail. Callers match on the variant.
#[derive(Debug, Clone, PartialEq)]
pub enum OrderError {
    Empty {
        order_id: String,
    },
    ZeroQuantity {
        order_id: String,
    },
    OutOfStock {
        order_id: String,
        name: String,
        available: u32,
        requested: u32,
    },
}

impl fmt::Display for OrderError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            OrderError::Empty { order_id } => write!(f, "{order_id}: order is
empty"),
            OrderError::ZeroQuantity { order_id } => {
                write!(f, "{order_id}: quantity must be at least 1")
            }
            OrderError::OutOfStock {
                order_id,
                name,
```

```
                available,
                requested,
            } => write!(
                f,
                "{order_id}: only {available} of {name} in stock, {requested}
requested"
            ),
        }
    }
}
```

Each way an order can fail is a variant carrying the details. A caller that needs to react differently to "out of stock" and "empty order" matches on the variant, and a test can check exactly which failure happened. The Python version would be an exception class hierarchy.

15.4 Discount Rules

`DiscountPolicy` is a trait with one method. Each discount rule is a struct that implements it. `OrderService` depends only on the trait, so a new rule needs a new struct and no change to `OrderService`.

src/orders/discounts.rs

```rust
pub trait DiscountPolicy {
    /// The discount for this order, in cents.
    fn discount_for(&self, order: &Order) -> i64;
}

pub struct NoDiscount;

impl DiscountPolicy for NoDiscount {
    fn discount_for(&self, _order: &Order) -> i64 {
        0
    }
}

/// A percentage off any order whose subtotal reaches a threshold.
pub struct BulkDiscount {
    pub threshold_cents: i64,
    pub percent: i64,
}

impl BulkDiscount {
    pub fn new(threshold_cents: i64, percent: i64) -> BulkDiscount {
        BulkDiscount {
            threshold_cents,
            percent,
        }
    }
}
```

```
// Default is Rust's version of default arguments: BulkDiscount::default()
impl Default for BulkDiscount {
    fn default() -> BulkDiscount {
        BulkDiscount::new(100_00, 10)
    }
}

impl DiscountPolicy for BulkDiscount {
    fn discount_for(&self, order: &Order) -> i64 {
        let subtotal = order.subtotal();
        if subtotal < self.threshold_cents {
            return 0;
        }
        percent_of(subtotal, self.percent)
    }
}
```

- `pub struct NoDiscount;` is a **unit struct**: no fields, so it takes no memory. It exists only to carry a trait implementation.
- `impl Default for BulkDiscount` is how Rust offers default values. Rust has no default arguments and no function overloading, so `BulkDiscount::default()` and `BulkDiscount::new(100_00, 10)` are two separate functions.
- `_order`: a leading underscore marks a parameter as deliberately unused, which silences the compiler's warning.

15.5 Checkout

`checkout` returns either `Ok(Receipt)` or `Err(OrderError)`. It checks every line before it removes any stock, so a failed order leaves the stock unchanged.

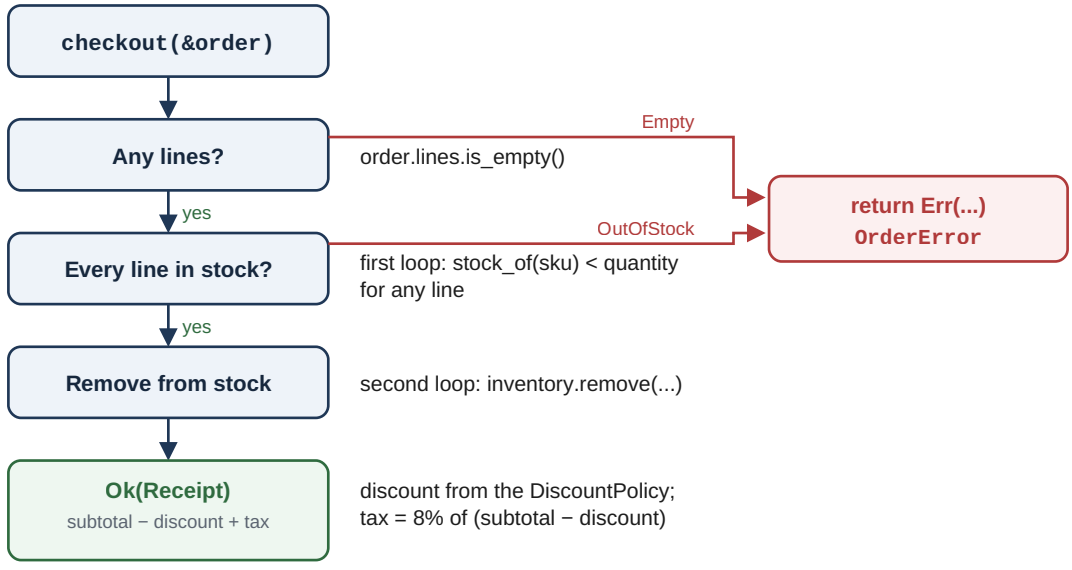


Figure 4. `OrderService::checkout`. Either check can return an error; only a fully valid order changes stock.

src/orders/service.rs

```rust
pub const TAX_RATE_PERCENT: i64 = 8;

/// How many of each product are on the shelf.
#[derive(Debug, Default)]
pub struct Inventory {
    stock: HashMap<String, u32>,
}

impl Inventory {
    pub fn new() -> Inventory {
        Inventory {
            stock: HashMap::new(),
        }
    }

    pub fn restock(&mut self, sku: &str, quantity: u32) {
        *self.stock.entry(sku.to_string()).or_insert(0) += quantity;
    }

    // copied() turns Option<&u32> into Option<u32>; a missing sku has 0
    pub fn stock_of(&self, sku: &str) -> u32 {
        self.stock.get(sku).copied().unwrap_or(0)
    }

    fn remove(&mut self, sku: &str, quantity: u32) {
        if let Some(count) = self.stock.get_mut(sku) {
            *count -= quantity;
        }
```

```
    }
}
```

src/orders/service.rs

```
/// Turns an Order into a Receipt. Generic over the discount rule D:
/// the caller chooses the rule, and the service never names a concrete one.
pub struct OrderService<D: DiscountPolicy> {
    inventory: Inventory,
    discounts: D,
}

impl<D: DiscountPolicy> OrderService<D> {
    // Dependency injection: the service receives its collaborators
    // instead of creating them. Tests pass in whatever they need.
    pub fn new(inventory: Inventory, discounts: D) -> OrderService<D> {
        OrderService {
            inventory,
            discounts,
        }
    }

    pub fn inventory(&self) -> &Inventory {
        &self.inventory
    }

    pub fn inventory_mut(&mut self) -> &mut Inventory {
        &mut self.inventory
    }

    pub fn checkout(&mut self, order: &Order) -> Result<Receipt, OrderError> {
        if order.lines.is_empty() {
            return Err(OrderError::Empty {
                order_id: order.id.clone(),
            });
        }

        // Check every line first, so a failure leaves the stock untouched.
        for line in &order.lines {
            let available = self.inventory.stock_of(&line.product.sku);
            if available < line.quantity {
                return Err(OrderError::OutOfStock {
                    order_id: order.id.clone(),
                    name: line.product.name.clone(),
                    available,
                    requested: line.quantity,
                });
            }
        }

        for line in &order.lines {
            self.inventory.remove(&line.product.sku, line.quantity);
        }

        let subtotal = order.subtotal();
        let discount = self.discounts.discount_for(order);
```

```
        let tax = percent_of(subtotal - discount, TAX_RATE_PERCENT);
        Ok(Receipt {
            order_id: order.id.clone(),
            subtotal_cents: subtotal,
            discount_cents: discount,
            tax_cents: tax,
            total_cents: subtotal - discount + tax,
        })
    }
}
```

- `copied().unwrap_or(0)`: `get` returns `Option<&u32>`, a reference into the map; `copied` turns it into `Option<u32>`, and a missing SKU counts as 0.
- `remove` has no `pub`, so only code in this module can take stock away. The service's `checkout` is the only path.
- `checkout` takes `&mut self` because it changes the inventory, and `&Order` because it only reads the order.
- `#[derive(Default)]` on `Inventory` sits beside a hand-written `new` because Clippy warns when a type has a public `new()` with no arguments but no `Default`.

15.6 Dependency Injection and Ownership

Dependency injection (DI) means a type receives the objects it works with instead of creating them. `OrderService::new` receives an `Inventory` and a discount rule; it never writes `BulkDiscount::default()` itself. The caller decides which rule applies, and a test can pass `NoDiscount`.

`OrderService<D: DiscountPolicy>` takes the rule as a **generic**, so each service is built for one concrete rule and calls it directly (section 11.2). Storing a `Box<dyn DiscountPolicy>` instead would allow changing the rule at run time, at the cost of dynamic dispatch. Both are common; generics are the default choice in Rust.

Ownership changes the design compared with Python or C#. There, the caller and the service could both hold the same inventory object. Here, `new` *moves* the inventory into the service, and the service is its only owner. Other code reaches it by borrowing through `inventory()` (read) or `inventory_mut()` (change). Who can change what is visible in the method signatures.

15.7 Running It

src/ch15_orders.rs

```
// in run():
let keyboard = Product::new("KB-01", "Keyboard", 45_00);
let mouse = Product::new("MS-02", "Mouse", 25_00);
let monitor = Product::new("MN-03", "Monitor", 180_00);

let mut inventory = Inventory::new();
inventory.restock(&keyboard.sku, 10);
inventory.restock(&mouse.sku, 10);
inventory.restock(&monitor.sku, 1);

// The service takes ownership of the inventory: it is moved in here.
let mut service = OrderService::new(inventory, BulkDiscount::default());

let mut small = Order::new("ORD-1", "Ada");
small.add(mouse.clone(), 2).unwrap();
print_receipt(&service.checkout(&small).unwrap());

let mut large = Order::new("ORD-2", "Grace");
large.add(keyboard.clone(), 1).unwrap();
large.add(monitor.clone(), 1).unwrap();
print_receipt(&service.checkout(&large).unwrap());

let mut too_many = Order::new("ORD-3", "Linus");
too_many.add(monitor.clone(), 1).unwrap(); // the only monitor went in ORD-2
match service.checkout(&too_many) {
    Ok(receipt) => print_receipt(&receipt),
    Err(e) => println!("Rejected: {e}"),
}

println!("Mice left: {}", service.inventory().stock_of(&mouse.sku));
```

Run it with `cargo run -- 15`:

Output

```
ORD-1: subtotal 50.00 - discount 0.00 + tax 4.00 = 54.00
ORD-2: subtotal 225.00 - discount 22.50 + tax 16.20 = 218.70
Rejected: ORD-3: only 0 of Monitor in stock, 1 requested
Mice left: 8
```

ORD-1 is below the 100.00 threshold, so it gets no discount. ORD-2 is 225.00, so it gets 10% off (22.50), and tax is 8% of the remaining 202.50. ORD-3 fails because ORD-2 bought the only monitor.

15.8 The Tests

Rust's test framework is built in: no package to install. A test is any function marked `#[test]`; it passes if it returns without panicking. There are two places to put tests:

- **Unit tests** sit in the same file as the code, inside a `#[cfg(test)] mod tests` block that is compiled only by `cargo test`. They can call private functions. `src/money.rs` and `src/orders/discounts.rs` have them.
- **Integration tests** sit in `tests/`. Each file is compiled as a separate program that can use only the library's public items, the way outside code would.

tests/order_service.rs

```
// Helper: a service with 5 of each product and the given discount rule
fn build<D: DiscountPolicy>(discounts: D) -> OrderService<D> {
    let mut stock = Inventory::new();
    stock.restock("MS-02", 5);
    stock.restock("MN-03", 5);
    OrderService::new(stock, discounts)
}

#[test]
fn checkout_not_enough_stock_fails_and_leaves_stock_alone() {
    let mut service = build(NoDiscount);
    let mut order = Order::new("T-3", "Linus");
    order.add(mouse(), 1).unwrap();
    order.add(monitor(), 99).unwrap();

    let result = service.checkout(&order);

    assert!(matches!(
        result,
        Err(OrderError::OutOfStock { available: 5, .. })
    ));
    assert_eq!(service.inventory().stock_of("MS-02"), 5); // the mouse was not
removed
}
```

Each test follows **Arrange, Act, Assert**: set up objects, call the method under test, check the result. `matches!(value, pattern)` is `true` if the value fits the pattern; here it checks the error variant and one field, ignoring the rest with `..`.

cargo test (abridged)

```
     Running unittests src/lib.rs
running 4 tests
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

     Running tests/bank_account.rs
running 4 tests
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

     Running tests/order_service.rs
running 4 tests
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

pytest	Rust
`def test_x():`	`#[test] fn x() {`
`assert actual == expected`	`assert_eq!(actual, expected);`
`assert cond`	`assert!(cond);`
`with pytest.raises(E):`	`assert!(result.is_err());` or `#[should_panic]`
`pytest -k name`	`cargo test name`
`@pytest.fixture`	a plain helper function, such as `build`
`pytest`	`cargo test`

Because errors are values, most failure tests check the returned `Err`, not a panic. `#[should_panic]` exists for code that is meant to panic. Code examples inside `///` doc comments are also compiled and run by `cargo test` ("doc tests"), so documentation examples cannot go stale.

16 Walkthrough: The Web API

Rust web back ends are usually built with **axum** (from the Tokio team) or **actix-web**. axum's style resembles FastAPI or Flask: you map a URL to an async function. The package in `api/` serves the order system over HTTP in about 120 lines. Its `Cargo.toml` lists `axum`, `tokio`, `serde`, `serde_json`, and the `tutorial` library by path.

16.1 Shared State

api/src/main.rs

```
// Everything the handlers share. Requests run on many threads at once, so
// anything that changes sits behind a Mutex, and the whole struct sits in
// an Arc so every handler can hold it.
struct AppState {
    catalog: BTreeMap<String, Product>,
    service: Mutex<OrderService<BulkDiscount>>,
    next_order: Mutex<u32>,
}

type SharedState = Arc<AppState>;
```

Requests are handled concurrently, on several threads. Every handler needs the catalog and the order service, and the service changes on every order. Chapter 14's rules decide the design:

- The whole state sits in an `Arc`, so every request can hold a pointer to it.
- What changes (the service, the order counter) sits in a `Mutex`; what never changes (the catalog) does not need one.
- `type SharedState = Arc<AppState>;` is a **type alias**, a short name for a long type.

Replace the `Mutex` with chapter 13's single-threaded `RefCell`, and the program does not compile: "`RefCell<OrderService<BulkDiscount>>` cannot be shared between threads safely". In Python the same mistake compiles, runs, and corrupts stock counts under load.

16.2 Start-up and Routes

api/src/main.rs

```
#[tokio::main]
async fn main() {
    // 1. Build the shared state.
    let mut catalog = BTreeMap::new();
```

```
    for product in [
        Product::new("KB-01", "Keyboard", 45_00),
        Product::new("MS-02", "Mouse", 25_00),
        Product::new("MN-03", "Monitor", 180_00),
    ] {
        catalog.insert(product.sku.clone(), product);
    }
    let mut inventory = Inventory::new();
    for sku in catalog.keys() {
        inventory.restock(sku, 5);
    }
    let state = Arc::new(AppState {
        catalog,
        service: Mutex::new(OrderService::new(inventory,
BulkDiscount::default())),
        next_order: Mutex::new(1000),
    });

    // 2. Map URLs to handler functions.
    let app = Router::new()
        .route("/products", get(list_products))
        .route("/stock/{sku}", get(stock_of))
        .route("/orders", post(create_order))
        .with_state(state);

    // 3. Listen and serve until stopped with Ctrl+C.
    let listener = tokio::net::TcpListener::bind("127.0.0.1:3000")
        .await
        .unwrap();
    println!("listening on http://{}", listener.local_addr().unwrap());
    axum::serve(listener, app).await.unwrap();
}
```

`#[tokio::main]` (the line above `main`) starts the Tokio runtime and runs the async `main` on it. `Router::new().route(path, get(handler))` is FastAPI's `@app.get(path)`. `{sku}` in a path is a parameter. `with_state` hands the shared state to every handler.

16.3 Handlers

api/src/main.rs

```
// axum fills each parameter from the request: State from the shared state,
// Path from the URL, Json from the request body.
async fn list_products(State(state): State<SharedState>) -> Json<Vec<Product>> {
    Json(state.catalog.values().cloned().collect())
}

async fn stock_of(State(state): State<SharedState>, Path(sku): Path<String>) ->
Json<Value> {
    let service = state.service.lock().unwrap();
    Json(json!({ "sku": sku, "in_stock": service.inventory().stock_of(&sku) }))
}
```

api/src/main.rs

```
// The JSON body of POST /orders. Deserialize builds it from JSON.
#[derive(Deserialize)]
struct LineRequest {
    sku: String,
    quantity: u32,
}

#[derive(Deserialize)]
struct OrderRequest {
    customer: String,
    lines: Vec<LineRequest>,
}
```

api/src/main.rs

```
// Returning a Result: Ok becomes 200 with a JSON receipt, Err becomes the
// given status code with a JSON error message.
async fn create_order(
    State(state): State<SharedState>,
    Json(request): Json<OrderRequest>,
) -> Result<Json<Receipt>, (StatusCode, Json<Value>)> {
    let order_id = {
        let mut next = state.next_order.lock().unwrap();
        *next += 1;
        format!("ORD-{next}")
    }; // the lock is released here, at the end of the block

    let mut order = Order::new(&order_id, &request.customer);
    for line in &request.lines {
        let Some(product) = state.catalog.get(&line.sku) else {
            let error = json!({ "error": format!("unknown SKU {}", line.sku) });
            return Err((StatusCode::NOT_FOUND, Json(error)));
        };
        order
            .add(product.clone(), line.quantity)
            .map_err(bad_request)?;
    }
```

```
    let mut service = state.service.lock().unwrap();
    let receipt = service.checkout(&order).map_err(bad_request)?;
    Ok(Json(receipt))
}

fn bad_request(error: OrderError) -> (StatusCode, Json<Value>) {
    (
        StatusCode::BAD_REQUEST,
        Json(json!({ "error": error.to_string() })),
    )
}
```

- **Extractors.** Each handler parameter says where its value comes from: `State(state)` from the shared state, `Path(sku)` from the URL, `Json(request)` from the JSON body. FastAPI reads type hints the same way. A body that does not fit `OrderRequest` is rejected with HTTP 422 before the handler runs.
- **Return types.** `Json(value)` sends the value as JSON with status 200. A `(StatusCode, Json(...))` pair sets the status. Returning `Result` lets the handler use `?`; an `Err` becomes the error response.
- `map_err(bad_request)?` converts an `OrderError` into a 400 response and returns it early. This replaces `try`/`except` around the business logic.
- `let Some(product) = ... else { return ...; };` is **let-else**: bind the value if the pattern matches, otherwise run the `else` block, which must leave the function.
- `json!({ ... })` builds a JSON value with Python-dict-like syntax.
- The order-number block `{ let mut next = ...; ... }` releases its lock at its closing brace (section 13.2), before checkout takes the other lock.

16.4 Trying It

Terminal 1

```
cargo run -p tutorial-api
listening on http://127.0.0.1:3000
```

Terminal 2

```
curl localhost:3000/products
[{"sku":"KB-01","name":"Keyboard","price_cents":4500},
{"sku":"MN-03","name":"Monitor","price_cents":18000},
{"sku":"MS-02","name":"Mouse","price_cents":2500}]

curl localhost:3000/stock/MN-03
{"in_stock":5,"sku":"MN-03"}

curl -X POST localhost:3000/orders -H 'Content-Type: application/json' \
     -d '{"customer":"Ada","lines":[{"sku":"MN-03","quantity":1}]}'
{"order_id":"ORD-1001","subtotal_cents":18000,"discount_cents":1800,"tax_cents":
1296,"total_cents":17496}

curl -X POST localhost:3000/orders -H 'Content-Type: application/json' \
     -d '{"customer":"Ada","lines":[{"sku":"XX-99","quantity":1}]}'
{"error":"unknown SKU XX-99"}                                  (HTTP 404)

curl -X POST localhost:3000/orders -H 'Content-Type: application/json' \
     -d '{"customer":"Ada","lines":[{"sku":"MN-03","quantity":9}]}'
{"error":"ORD-1003: only 4 of Monitor in stock, 9 requested"}   (HTTP 400)

curl -X POST localhost:3000/orders -H 'Content-Type: application/json' \
     -d '{"customer":"Ada","lines":[{"sku":"MN-03","quantity":-1}]}'
Failed to deserialize the JSON body into the target type: lines[0].quantity:
invalid value: integer `-1`, expected u32 at line 1 column 55     (HTTP 422)
```

The outputs above come from one fresh server, in this order. Order numbers are taken even by orders that fail, which is why the third order is ORD-1003. The products come out sorted by SKU because the catalog is a `BTreeMap`. Stop the server with Ctrl+C; the stock resets on the next start, because it lives in memory.

Production services add a database through **sqlx** (async SQL with queries checked at compile time) or **Diesel** (an ORM), and logging through the **tracing** crate.

17 Hands-on Drills

Each drill is a small change to the companion project, ordered from easiest to hardest. Run the stated command after each change. Solutions are in appendix A. Before starting, check that the project is clean: `cargo test` should report 4 passed in each of its three test groups (12 in all).

> **Undo.** The project is a git repository. After a drill, `git checkout -- .` restores every file to its committed state. To keep your answers instead, work on a branch: `git switch -c drills`.

17.1 Drill 1: Read a Failing Test

In `src/orders/service.rs`, change `TAX_RATE_PERCENT` from `8` to `10`.

- Predict the new totals for ORD-1 and ORD-2, then check with `cargo run -- 15`.
- Run `cargo test`. Two tests fail. Find the `left` and `right` lines in the output and explain each number. `assert_eq!(a, b)` calls its first argument "left" and its second "right".
- Change the rate back.

17.2 Drill 2: Read Compiler Errors

Make each change to `src/ch03_basics.rs` on its own, run `cargo build`, read the whole error, and undo it. The first line gives the error code and message; the `-->` line gives `file:line:column`.

Change	Error you will see
Delete the `;` after `let year = 1843`	`error: expected ``;``, found ``println```
Change `let mut count = 0;` to `let count = 0;`	`error[E0384]: cannot assign twice to immutable variable ``count```
Change `square(7)` to `square(7.5)`	`error[E0308]: mismatched types` ... `expected ``i32``, found floating-point number`
Add `;` after `n * n` in `square`	`error[E0308]: mismatched types` ... `expected ``i32``, found ``()``` , with `help: remove this semicolon to return this value`

Change	Error you will see
Change `println!("count is` to `printline!("count is`	`error: cannot find macro `printline` in this scope`, with `help: a macro with a similar name exists`

Then try the two ownership errors from chapter 6: remove the `//` in front of `println!("{a}")`, and separately in front of `scores.push(40)`, in `src/ch06_ownership.rs`.

17.3 Drill 3: Find a Bug with a Test

`BankAccount::withdraw` accepts a negative amount, and `withdraw(-50_00)` adds money. `deposit` rejects amounts that are not positive, but `withdraw` does not.

1. In `tests/bank_account.rs`, add a `#[test]` named `withdraw_negative_is_rejected` that deposits 100.00, checks that `withdraw(-50_00)` returns an error, and checks that the balance is still 100.00.
2. Run `cargo test` and watch the new test fail.
3. Fix `withdraw` and watch it pass. This order (failing test, then fix) is test-driven development.

Then think about the alternative: if the amount parameter were a `u64`, a negative amount could not be passed at all. Choosing types that make bad states impossible is a common Rust design argument.

17.4 Drill 4: Add a Method

Add `pub fn transfer_to(&mut self, other: &mut BankAccount, cents: i64) -> Result<(), String>` to `BankAccount`. It moves money from this account to `other`. Add a test that transfers 40.00 from an account holding 100.00 and checks both balances. Hint: reuse `withdraw` and `deposit`, and use `?`.

Afterwards, try calling `account.transfer_to(&mut account, 10_00)` with the same account on both sides. Explain the compiler's answer using section 6.4's rule.

17.5 Drill 5: A New Enum Variant

In `src/ch08_enums.rs`, add a `Triangle { base_len: f64, height: f64 }` variant to `Shape`, and nothing else. Run `cargo build` and read the two errors: the compiler lists every `match` that does not yet handle a triangle. Fix both (area is half of

base times height), then add `Shape::Triangle { base_len: 3.0, height: 4.0 }` to the `shapes` list. Expected new output lines:

Output

```
Triangle with area 6.00
Total area: 19.14
```

17.6 Drill 6: An Iterator Query

In `src/ch12_iterators.rs`, print the highest-paid employee in each department, departments in alphabetical order. Expected output:

Output

```
Top in Engineering: Grace
Top in Finance: Barbara
Top in Sales: Alan
```

Hint: loop over `&staff`, keeping a `BTreeMap` from department name to the best `&Employee` so far. `entry(...).or_insert(e)` gives you the current best to compare against.

17.7 Drill 7: A New Discount Rule

In `src/orders/discounts.rs`, write `LoyaltyDiscount`, a `DiscountPolicy` that gives 5% off to customers whose names are in a set passed to its constructor: `LoyaltyDiscount::new(&["Ada"])`. Put it above the `#[cfg(test)]` module (Clippy objects to items after the tests), and add it to the `pub use discounts::{...}` line in `src/orders/mod.rs`. In `src/ch15_orders.rs`, pass it instead of `BulkDiscount::default()`. Do not change `OrderService`. Expected:

Output

```
ORD-1: subtotal 50.00 - discount 2.50 + tax 3.80 = 51.30
ORD-2: subtotal 225.00 - discount 0.00 + tax 18.00 = 243.00
```

Afterwards, say in one sentence why `OrderService` did not need to change. That sentence is an interview answer about traits and dependency injection.

17.8 Drill 8: A New Endpoint

In `api/src/main.rs`, add `POST /stock/{sku}/{quantity}`, which restocks a product and returns the new stock level. Register the route next to the others, and write

the handler like `stock_of`. Restart the server, then test it (a fresh server holds 5 monitors):

Output

```
curl -X POST localhost:3000/stock/MN-03/10
{"in_stock":15,"sku":"MN-03"}
```

Hint: two path parameters arrive together as a tuple: `Path((sku, quantity)): Path<(String, u32)>`.

18 Interview Quick Reference

Each answer below is short enough to say aloud. Each one points to the example in the companion project that you can describe.

18.1 Likely Questions

How does Rust guarantee memory safety without a garbage collector?

Ownership. Every value has one owner, and it is freed when the owner goes out of scope. Values can be borrowed through references, with either many shared references or one mutable reference at a time, and no reference may outlive its value. The compiler's borrow checker enforces these rules, so use-after-free, double free and data races do not compile. (Chapter 6.)

What is the difference between a move, a copy and a clone?

A move transfers ownership; the old variable can no longer be used. Types that implement `Copy` (numbers, `bool`, `char`) are copied bit for bit on assignment instead, and both stay usable. `clone()` is an explicit, possibly expensive deep copy. Rust never clones implicitly. (Section 6.2.)

What is a lifetime?

The span of code during which a reference is valid. The compiler infers most lifetimes. When a function returns a reference derived from several inputs, you annotate the relationship, as in `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str`. Annotations describe constraints; they never extend how long anything lives. (Section 13.1.)

String versus &str?

`String` owns heap-allocated, growable UTF-8 text. `&str` is a borrowed view of text owned elsewhere, such as a `String` or a literal. Take `&str` in function parameters; store `String` in structs. (Section 4.2.)

How does Rust handle errors? Why no exceptions?

Recoverable errors are values: functions return `Result<T, E>`, and the `?` operator passes an error up to the caller. The possible failure is part of the signature, so it cannot be ignored by accident. Bugs cause a `panic`, which stops the thread. Missing values use `Option<T>` instead of null. `OrderService::checkout` returning `Result<Receipt, OrderError>` is the example. (Chapter 9.)

unwrap versus expect versus ?

`unwrap` panics on an error; `expect` panics with your message; `?` returns the error to the caller. Use `?` in library and application code, `expect` where an error is truly impossible or in `main`, and `unwrap` mainly in tests.

What is a trait? How does Rust do object-oriented programming?

A trait is a set of methods a type promises to implement, like an interface, and it can supply default methods. Rust has encapsulation (private fields, `pub`), polymorphism (traits) and abstraction, but no inheritance between structs: shared behaviour comes from traits and shared data from composition. `DiscountPolicy` with `NoDiscount` and `BulkDiscount` is the example. (Chapter 11.)

Generics versus trait objects? Static versus dynamic dispatch?

A generic `fn f<T: Trait>` is compiled separately for each concrete type (monomorphisation): static dispatch, fastest, but one type per use. A trait object `Box<dyn Trait>` calls through a vtable at run time: dynamic dispatch, slightly slower, but one collection can hold different types. `OrderService<D: DiscountPolicy>` uses a generic; `Vec<Box<dyn Describe>>` uses trait objects. (Sections 11.2, 11.3.)

Enums and match: why are they important?

Rust enums are sum types: a value is exactly one variant, and each variant can carry different data. `match` must handle every variant, so adding a variant turns every place that forgot it into a compile error. `Option`, `Result` and `OrderError` are all enums. (Chapter 8.)

Box, Rc, Arc, RefCell, Mutex: when do you use each?

`Box`: one owner, value on the heap (recursive types, trait objects). `Rc`: several owners in one thread. `Arc`: several owners across threads. `RefCell`: mutation through a shared reference in one thread, checked at run time. `Mutex`: the same across threads, one at a time. The web API's `Arc<AppState>` holding a `Mutex<OrderService>` is the example. (Sections 13.4, 16.1.)

What are Send and Sync? What is fearless concurrency?

`Send`: a value can move to another thread. `Sync`: a value can be shared between threads by reference. The compiler implements them automatically where safe and refuses to share anything else across threads, so data races become compile errors. `Rc` is neither; `Arc` is both. (Section 14.2.)

How does async work in Rust?

`async fn` returns a future, a value describing work that does nothing until awaited. The standard library has no runtime to drive futures; Tokio is the usual choice. `tokio::join!` runs futures concurrently, like `asyncio.gather`. Use async for I/O-bound work with many waits, threads for CPU-bound work. (Section 14.4.)

What does unsafe do?

An `unsafe` block allows a few operations the compiler cannot check, such as dereferencing raw pointers or calling C functions. It does not turn the borrow checker off for ordinary code. Safe wrappers around small `unsafe` cores are how the standard library builds `Vec` and `Mutex`. Application code rarely needs it.

What are macros? Why does println! have a "!"?

Macros generate code at compile time. `println!` checks its format string against its arguments during compilation; `vec!` builds a vector; `#[derive(Debug)]` writes trait implementations. The `!` marks a macro call as different from a function call.

What are zero-cost abstractions?

High-level features that compile to the same machine code you would write by hand: iterator chains become plain loops, generics become specialised functions, and `Option<&T>` takes no more space than a pointer. You do not pay at run time for code you do not use. (Section 12.4.)

Why would a Python team use Rust?

For CPU-bound hot spots and for tools where speed and reliability matter. PyO3 and maturin build Rust code into a Python extension module, which is how `pydantic-core` and `polars` work. Tools such as `uv` and `ruff` are Rust programs. The trade-off is a slower edit-compile cycle and a steeper learning curve.

How would you test this code?

With Rust's built-in test framework: `#[test]` functions run by `cargo test`, unit tests in a `#[cfg(test)]` module beside the code, integration tests in `tests/`. Inject collaborators through constructors or generics so a test can pass a simple version, as the order tests pass `NoDiscount`. (Section 15.8.)

18.2 Talking About Your Background

> **Suggested answer**

> Say that your strongest language is Python, and that you have studied Rust and can read it and make changes. Name the mappings you know: dataclasses to structs with `derive`, exceptions to `Result` and `?`, `None` to `Option`, ABCs and Protocols to traits, comprehensions to iterator chains, `with` to `Drop`, `asyncio` to Tokio, pytest to `cargo test`. Say that ownership is the new idea for you, and then explain it correctly in two sentences. Interviewers trust a precise explanation of a small thing more than a broad claim.

18.3 Reading Unfamiliar Rust Code

1. Open `Cargo.toml`: the edition, the dependencies (they tell you the stack: `tokio`, `axum`, `serde`, `sqlx`), and `[workspace]` members.
2. Find the entry point: `src/main.rs` and its `fn main`, or `src/lib.rs` for a library. The `mod` lines list the modules.
3. Read the types before the functions: `struct`s, `enum`s and `trait`s are the design. An error enum lists everything that can go wrong.
4. For a function, read the signature first. `&self`, `&mut self` or `self` tells you whether it reads, changes or consumes; a `Result` return tells you it can fail.
5. Follow the `?`s: each one is a place the function can return early with an error.
6. In a web service, find the `Router`: it lists every endpoint and its handler.
7. Read the tests. Test names state the intended behaviour in plain words.
8. Run `cargo doc --open` for a browsable view of every public item.

18.4 Glossary

Term	Meaning
Borrow checker	The part of the compiler that enforces ownership and borrowing rules.
Crate	A compilation unit: one library or one program. Also a package on crates.io.
Derive	An attribute that generates a trait implementation: `#[derive(Debug)]`.
Drop	Freeing a value when its owner goes out of scope; the trait that runs cleanup code then.
Edition	A set of language rules a crate opts into: 2015, 2018, 2021, 2024.
Lifetime	The span in which a reference is valid; written `'a`.
Macro	Code that generates code at compile time; called with `!`.
Module	A namespace, normally one file, declared with `mod`.

Term	Meaning
Monomorphisation	Compiling a separate copy of generic code for each concrete type.
Panic	An unrecoverable error that stops the thread; for bugs.
RAII	Tying a resource's release to a value's scope, through `Drop`.
Serde	The standard serialisation framework (JSON and many other formats).
Slice	A borrowed view of a contiguous sequence: `&[T]`, `&str`.
Trait object	A value of type `dyn Trait` behind a pointer, called through a vtable.
Turbofish	The `::<T>` syntax naming a type parameter: `parse::<u32>()`.
Unit type	`()`, the empty tuple: what a function with no return value returns.

A Drill Solutions

Each solution below was applied to a copy of the project and built. With all of them in place, all 14 tests pass, Clippy reports no warnings, and the outputs match those stated in chapter 17.

A.1 Drill 1

ORD-1: tax 5.00, total 55.00. ORD-2: tax 20.25, total 222.75. `cargo test` reports:

Output

```
thread 'checkout_large_order_gets_bulk_discount' panicked at tests/
order_service.rs:43:5:
assertion `left == right` failed
  left: 17820
 right: 17496
thread 'checkout_small_order_adds_tax_only' panicked at tests/order_service.rs:
30:5:
assertion `left == right` failed
  left: 500
 right: 400
test result: FAILED. 2 passed; 2 failed; 0 ignored; 0 measured; 0 filtered out
```

17820 cents is (180.00 − 18.00) × 1.10 = 178.20; 500 cents is 10% of 50.00. The tests expect the 8% results, 174.96 and 4.00, so they fail when the rate changes.

A.2 Drills 3 and 4

Before the fix, the new test fails with `assertion failed: account.withdraw(-50_00).is_err()`.

src/ch07_structs.rs

```
pub fn withdraw(&mut self, cents: i64) -> Result<(), String> {
    if cents <= 0 {
        return Err(format!("withdrawal must be positive, got {cents}"));
    }
    if cents > self.balance_cents {
        // ... unchanged ...
    }
    self.balance_cents -= cents;
    Ok(())
}

pub fn transfer_to(&mut self, other: &mut BankAccount, cents: i64) -> Result<(),
String> {
    self.withdraw(cents)?;
    other.deposit(cents)
}
```

tests/bank_account.rs

```
#[test]
fn withdraw_negative_is_rejected() {
    let mut account = BankAccount::new("ACC-1", "Ada");
    account.deposit(100_00).unwrap();
    assert!(account.withdraw(-50_00).is_err());
    assert_eq!(account.balance(), 100_00);
}

#[test]
fn transfer_moves_money() {
    let mut source = BankAccount::new("ACC-1", "Ada");
    let mut target = BankAccount::new("ACC-2", "Grace");
    source.deposit(100_00).unwrap();

    source.transfer_to(&mut target, 40_00).unwrap();

    assert_eq!(source.balance(), 60_00);
    assert_eq!(target.balance(), 40_00);
}
```

`transfer_to` withdraws first, so a failed withdrawal returns early through `?` and nothing is deposited. Transferring an account to itself needs `&mut self` and `&mut account` at once, two mutable borrows of one value, which the borrow checker refuses (E0499).

A.3 Drill 5

Adding the variant alone gives two errors, one per `match`:

Output

```
error[E0004]: non-exhaustive patterns: `&Shape::Triangle { .. }` not covered
  --> src/ch08_enums.rs:17:15
error[E0004]: non-exhaustive patterns: `&Shape::Triangle { .. }` not covered
  --> src/ch08_enums.rs:25:15
```

src/ch08_enums.rs

```
pub enum Shape {
    // ... Circle, Rectangle, Square unchanged ...
    Triangle { base_len: f64, height: f64 },
}

// in area():
Shape::Triangle { base_len, height } => 0.5 * base_len * height,

// in name():
Shape::Triangle { .. } => "Triangle",

// in run(), the last item of the shapes list:
Shape::Triangle { base_len: 3.0, height: 4.0 },
```

A.4 Drill 6

src/ch12_iterators.rs

```
use std::collections::BTreeMap;   // at the top of the file

// in run():
let mut top: BTreeMap<&str, &Employee> = BTreeMap::new();
for e in &staff {
    let best = top.entry(e.dept.as_str()).or_insert(e);
    if e.salary > best.salary {
        *best = e;
    }
}
for (dept, e) in &top {
    println!("Top in {dept}: {}", e.name);
}
```

`best` is a `&mut &Employee`: a mutable reference to the map's slot. `*best = e` replaces the reference stored in the slot. The `BTreeMap` gives alphabetical order; a `HashMap` would print the departments in a different order on each run.

A.5 Drill 7

src/orders/discounts.rs

```
use std::collections::HashSet;   // at the top of the file

/// 5% off for named loyal customers.
pub struct LoyaltyDiscount {
    customers: HashSet<String>,
}

impl LoyaltyDiscount {
    pub fn new(customers: &[&str]) -> LoyaltyDiscount {
        let mut set = HashSet::new();
        for name in customers {
            set.insert(name.to_string());
        }
        LoyaltyDiscount { customers: set }
    }
}

impl DiscountPolicy for LoyaltyDiscount {
    fn discount_for(&self, order: &Order) -> i64 {
        if self.customers.contains(&order.customer) {
            percent_of(order.subtotal(), 5)
        } else {
            0
        }
    }
}
```

src/ch15_orders.rs

```
use crate::orders::{Inventory, LoyaltyDiscount, Order, OrderService, Product,
Receipt};

// in run():
let mut service = OrderService::new(inventory, LoyaltyDiscount::new(&["Ada"]));
```

Remove `BulkDiscount` from the `use` line, or the compiler warns that it is unused. The one-sentence answer: `OrderService` is generic over any type that implements the `DiscountPolicy` trait, so a new rule plugs in without changing it.

A.6 Drill 8

api/src/main.rs

```
// in main(), with the other routes:
.route("/stock/{sku}/{quantity}", post(restock))

// a new handler:
async fn restock(
    State(state): State<SharedState>,
    Path((sku, quantity)): Path<(String, u32)>,
) -> Json<Value> {
    let mut service = state.service.lock().unwrap();
    service.inventory_mut().restock(&sku, quantity);
    Json(json!({ "sku": sku, "in_stock": service.inventory().stock_of(&sku) }))
}
```

A quantity that is not a whole number, such as `/stock/MN-03/ten`, fails to parse as a `u32`, and axum answers with HTTP 400 before the handler runs.